

The **did:webvh** DID Method v1.0



Specification Status: v1.0

This is the specification of the **did:webvh** DID Method, Version 1.0. Please note that we continue to make cleanups (e.g., fixing typos, broken links, missing references, etc.) and making wording clarifications in this version of the specification. With that work there will be no changes to the meaning of the specification.

The current [Editor's Draft](#) of the specification will be maintained in sync with this version as the cleanups and clarifications to v1.0 are made. When a change to the meaning of the specification is identified, the Editor's Draft synchronization with this version will be stopped, and this message will be updated.

Current Specification: [v1.0](#)

Specification Version: v1.0 (see [Changelog](#))

Source of Latest Draft:  <https://github.com/decentralized-identity/didwebvh>

Previous Versions:

- [v0.5](#)
- [v0.4](#)
- [v0.3](#)

Information Site: <https://didwebvh.info/>

Editors:

-  [Stephen Curran](#)
-  [John Jordan, BC Gov](#)
-  [Andrew Whitehead](#)
-  [Brian Richter](#)
-  [Michel Sahli](#)
-  [Martina Kolpondinos](#)
-  [Dmitri Zagdulin](#)
-  [Alexander Shenshin](#)

Participate:

-  [GitHub repo](#)
-  [File a bug](#)
-  [Commit history](#)

Implementations:

-  [TypeScript](#)
-  [Python](#)
-  [Go](#) - supports v03
-  [Rust](#) - work in progress
-  [did:webvh Server](#)
- [did:webvh ACA-Py Plugin](#)

§ 1. Abstract

DID Web + Verifiable History ([did:webvh](#)) is an enhancement to the [did:web](#) DID method, providing complementary features that address [did:web](#) 's limitations as a long-lasting DID. [did:webvh](#) features include:

- The same DID-to-HTTPS transformation as [did:web](#) .
- Ongoing publishing of the full history of the DID, including all of the DID Document ([DIDDoc](#)) versions instead of, or alongside an existing [did:web](#) DIDDoc.

- The ability to resolve the full history of the DID using a verifiable chain of updates to the [DIDDoc](#) from genesis to deactivation.
- A [self-certifying identifier](#) (SCID) for the DID. The [SCID](#), globally unique and embedded in the DID, is derived from the initial [DID log entry](#). It ensures the integrity of the DID's history mitigating the risk of attackers creating a new object with the same identifier.
- An optional mechanism for enabling [DID portability](#) via the [SCID](#), allowing the DID's web location to be moved and the DID string to be updated, both while retaining a connection to the predecessor DID(s) and preserving the DID's verifiable history.
- [DIDDoc](#) updates contain a proof signed by the [DID Controller's](#) *authorized* to update the DID.
- An optional mechanism for publishing "pre-rotation" keys to prevent the loss of control of a DID in cases where an active private key is compromised.
- An optional mechanism for having collaborating [witnesses](#) that approve of updates to the DID by a [DID Controller](#) before publication.
- Support for cryptographic agility through versioned specification upgrades, algorithm-identifying formats (e.g., [multihash](#) and [Data Integrity](#) proofs), and per-entry method parameters in the [DID log](#) that enable DIDs to evolve cryptographically over time.
- An optional mechanism for publishing the location of [did:webvh watchers](#) in the [DID log](#) that resolvers can use as another source DID data for long term resolution or detection of malicious [DID Controllers](#).
- DID URL path handling that defaults (but can be overridden) to automatically resolving [<did>/path/to/file](#) by using a comparable DID-to-HTTPS translation as for the [DIDDoc](#).
- A DID URL path [<did>/whois](#) that defaults to automatically returning (if published by the [DID controller](#)) a [Verifiable Presentation](#) containing [Verifiable Credentials](#) with the DID as the [credentialSubject](#), signed by the DID. It draws inspiration from the traditional WHOIS protocol [\[RFC3912\]](#), offering an easy-to-use, decentralized, trust registry.

Combined, the additional features enable greater trust, security and verifiability without compromising the simplicity of [did:web](#).

For information beyond this specification about the ([did:webvh](#)) DID method and how (and where) it is used in practice, please visit <https://didwebvh.info/>

§ 2. Overview

The emergence of [Decentralized Identifiers](#) (DIDs) and with them the evolution of [DID Methods](#) continues to be a dynamic area of development in the quest for trusted, secure and private digital identity management where the users are in control of their own data.

The `did:web` method, for example, leverages the Domain Name System (DNS) to perform the DID operations. This approach is praised for its simplicity and ease of deployment, including DID-to-HTTPS transformation and addressing some aspects of trust by allowing for DIDs to be associated with a domain's reputation or published on platforms such as GitHub. However, it is not without its challenges— from trust layers inherited from the web and the absence of a verifiable history for the DID.

Tackling these concerns, the `did:webvh` (`did:web` + Verifiable History) DID Method aims to enhance `did:web` by introducing features such as a [self-certifying identifiers](#) (SCIDs), update key(s) and a verifiable history, akin to what is available with ledger-based DIDs, but without relying on a ledger.

This approach not only maintains backward compatibility but also offers an additional layer of assurance for those requiring more robust verification processes. By publishing the resulting DID as both `did:web` and `did:webvh`, it caters to a broader range of trust requirements, from those who are comfortable with the existing `did:web` infrastructure to those seeking greater security assurances provided by `did:webvh`. This innovative step represents a significant stride towards a more trusted and secure web, where the integrity of cryptographic key publishing is paramount.

The key differences between `did:web` and `did:webvh` revolve around the core issues of decentralization and security. `did:web` is recognized for its simplicity and cost-effectiveness, allowing for easy establishment of a credential ecosystem. However, it is not inherently decentralized as it relies on DNS domain names, which require centralized registries. Furthermore, it lacks a cryptographically verifiable, tamper-resistant, and persistently stored DID document. In contrast, `did:webvh` is an enhancement to `did:web`, aiming to address these limitations by adding a verifiable history to the DID without the need for a ledger. This method provides a more decentralized approach by ensuring that the security of the embedded SCID does not depend on DNS. `did:webvh` is capable of resolving a cryptographically verifiable trust registry and status lists, using DID-Linked Resources, which `did:web` lacks. These features are designed to build a trusted web by offering a higher level of assurance for cryptographic key publishing and management.

For backwards compatibility, and for verifiers that “trust” `did:web`, a `did:webvh` can be trivially modified and published with a parallel `did:web` DID. For resolvers that want more assurance, `did:webvh` provides a way to verify a `did:web` using the features listed in the [Abstract](#).

The following is a `tl;dr` summary of how `did:webvh` works:

1. `did:webvh` uses the same DID-to-HTTPS transformation as `did:web`, so `did:webvh`'s `did.jsonl` ([JSON Lines](#)) file is found in the same location as `did:web`'s `did.json` file, and supports an easy transition from `did:web` to gain the added benefits of `did:webvh`.

2. The `did.jsonl` is a list of JSON [DID log entries](#), one per line, whitespace removed (per [JSON Lines](#)). Each entry contains the information needed to derive a version of the [DIDDoc](#) from its preceding version. The `did.jsonl` is also referred to as the [DID Log](#).
3. Each [DID log entry](#) is a JSON object containing the following properties:
 1. `versionId` – a value that combines the version number (starting at `1` and incremented by one per version), a literal dash `-`, and a hash of the entry. The [entry hash](#) calculation links each entry to its predecessor in a ledger-like chain.
 2. `versionTime` – as asserted by the [DID Controller](#).
 3. `parameters` – a set of [parameters](#) that impact the processing of the current and future [log entries](#).
 - Example [parameters](#) are the version of the `did:webvh` specification and hash algorithm being used, as well as the [SCID](#) and update key(s).
 4. `state` – the new version of the [DIDDoc](#).
 5. A [Data Integrity](#) (DI) proof across the entry, signed by a [DID Controller](#)-authorized key to update the [DIDDoc](#).
 6. If the [DID Controller](#) enables support for DID `[[witnesses]]`, an extra file (`did-witness.json`) in the same web location contains [Data Integrity](#) proofs from witness for [DID Log entries](#).
4. In generating the first version of the [DIDDoc](#), the [DID](#) calculates the [SCID](#) for the DID from the first [log entry](#) (which includes the [DIDDoc](#)). This is done by using the string `"{SCID}"` everywhere the actual [SCID](#) is to be placed in order to generate the hash. The [DID Controller](#) then replaces the placeholders with the calculated [SCID](#), including it as a `parameter` in the first [log entry](#), and inserting it where needed in the initial (and all subsequent) [DIDDocs](#). The [SCID](#) must be verified by the resolvers, to verify that the inception event has not been tampered with. The [SCID](#) also enables an optional [portability](#) capability, allowing a DID's web location to be moved, while retaining the [SCID](#) and verifiable history of the identifier.
5. A [DID Controller](#) generates and publishes the new/updated [DID](#) file by making it available at the appropriate location on the web, based on the DID's identifier. If a `did:webvh` has [watchers](#), a webhook is triggered to notify the [watchers](#) that an update is available and should be retrieved.
6. Given a `did:webvh` DID, a resolver converts the DID to an HTTPS URL, retrieves, and processes the [DID Log](#) `did.jsonl` file, generating and verifying each [log entry](#) as per the requirements outlined in this specification.
 - In the process, the resolver collects all the [DIDDoc](#) versions and public keys used by the DID currently, and in the past. This enables resolving both current and past versions of the DID and keys.
7. `did:webvh` DID URLs with paths and `/whois` are resolved to documents published by the [DID Controller](#) that are by default in the web location relative to the `did.jsonl` file.

See the [note below](#) about the powerful capability enabled by the `/whois` DID URL path.

8. A [DID Controller](#) can easily generate and publish a `did:web` DIDDoc from the latest `did:webvh` [DIDDoc](#) in parallel with the `did:webvh` [DID Log](#).

WARNING

A resolver settling for just the ``did:web`` version of the DID does not get the verifiability of the ``did:webvh`` log.

An example of a `did:webvh` evolving through a series of versions can be seen in the [did:webvh Examples](#) on the `did:webvh` information site.

§2.1 The `/whois` Use Case

The `did:webvh` DID Method introduces what we hope will be a widely embraced convention for all [DID Methods](#) – the `/whois` path. This feature harkens back to the `WHOIS` protocol that was created in the 1970s to provide a directory about people and entities in the early days of ARPANET. In the 80's, `whois` evolved into [\[RFC920\]](#) that has expanded into the [global whois](#) feature we know today as [\[RFC3912\]](#). Submit a `whois` request about a domain name, and get back the information published about that domain.

We propose that the `/whois` path for a DID enable a comparable, decentralized, version of the `WHOIS` protocol for DIDs. Notably, when `<did>/whois` is resolved (using a standard DID `service` that follows the [Linked-VP](#) specification), a [Verifiable Presentation](#) (VP) may be returned (if published by the [DID Controller](#)) containing [Verifiable Credentials](#) with the DID as the `credentialSubject`, and the VP signed by the DID. Given a DID, one can gather verifiable data about the [DID Controller](#) by resolving `<did>/whois` and processing the returned VP. That's powerful – an efficient, highly decentralized, trust registry. For `did:webvh`, the approach is very simple – transform the DID to its HTTPS equivalent, and execute a `GET <https>/whois`. Need to know who issued the VCs in the VP? Get the issuer DIDs from those VCs, and resolve `<issuer did>/whois` for each. This is comparable to walking a CA (Certificate Authority) hierarchy, but self-managed by the [DID Controllers](#) – and the issuers that attest to them.

The following is a use case for the `/whois` capability. Consider an example of the `did:webvh` controller being a mining company that has exported a shipment and created a “Product Passport” Verifiable Credential with information about the shipment. A country importing the shipment (the Importer) might want to know more about the issuer of the VC, and hence, the details of the shipment. They resolve the `<did>/whois` of the entity and get back a Verifiable Presentation about that DID. It might contain:

- A verifiable credential issued by the Legal Entity Registrar for the jurisdiction in which the mining company is headquartered.

- Since the Importer knows about the Legal Entity Registrar, they can automate this lookup to get more information about the company from the VC – its legal name, when it was registered, contact information, etc.
- A verifiable credential for a “Mining Permit” issued by the mining authority for the jurisdiction in which the company operates.
 - Perhaps the Importer does not know about the mining authority for that jurisdiction. The Importer can repeat the `/whois` resolution process for the issuer of *that* credential. The Importer might (for example), resolve and verify the `did:webvh` DID for the Authority, and then resolve the `/whois` DID URL to find a verifiable credential issued by the government of the jurisdiction. The Importer recognizes and trusts that government’s authority, and so can decide to recognize and trust the mining permit authority.
- A verifiable credential about the auditing of the mining practices of the mining company. Again, the Importer doesn’t know about the issuer of the audit VC, so they resolve the `/whois` for the DID of the issuer, get its VP and find that it is accredited to audit mining companies by the [London Metal Exchange](#) according to one of its mining standards. As the Importer knows about both the London Metal Exchange and the standard, it can make a trust decision about the original Product Passport Verifiable Credential.

Such checks can all be done with a handful of HTTPS requests and the processing of the DIDs and verifiable presentations. If the system cannot automatically make a trust decision, lots of information has been quickly collected that can be passed to a person to make such a decision.

The result is an efficient, verifiable, credential-based, decentralized, multi-domain trust registry, empowering individuals and organizations to verify the authenticity and legitimacy of DIDs. The convention promotes a decentralized trust model where trust is established through cryptographic verification rather than reliance on centralized authorities. By enabling anyone to access and validate the information associated with a DID, the “/whois” path contributes to the overall security and integrity of decentralized networks.

§ 3. `did:webvh` DID Method Specification

§ 3.1 Target System

The target system of the `did:webvh` DID method is the host (or domain) name when the domain specified by the DID is resolved through the Domain Name System (DNS) and verified by processing a log of DID versions.

§ 3.2 Method Name

The namestring that identifies this DID method is: `webvh`. A DID that uses this method **MUST** begin with the following prefix: `did:webvh`. Per the DID specification, this string **MUST** be in

lowercase. The remainder of the DID, after the prefix, is the [method-specific identifier](#), specified below.

§ 3.3 Method-Specific Identifier

The `did:webvh` method-specific identifier contains both the [self-certifying identifier](#) (SCID) for the DID, and a fully qualified domain name (with an optional path) that is secured by a TLS/SSL certificate. Given the DID, a [transformation to an HTTPS URL](#) is performed such that the [DID Log](#) for the `did:webvh` DID can be retrieved (via an `HTTP GET`) and processed to produce the [DIDDoc](#) for the DID. As per the Augmented Backus-Naur Form (ABNF) notation below, the [SCID](#) **MUST** be the first element of the method-specific identifier.

Formal rules describing valid domain name syntax are described in [\[RFC1035\]](#), [\[RFC1123\]](#), and [\[RFC2181\]](#). Each `did:webvh` DID's globally unique [SCID](#) **MUST** be [generated](#) during the creation of the DID based on its initial content and placed into the DID identifier for publication and use.

The domain name element of the method-specific identifier **MUST** match the name found in the SSL/TLS certificate per [\[RFC6125\]](#) and the its replacement [\[RFC9525\]](#), and it **MUST NOT** include IP addresses. A port **MAY** be included and the colon **MUST** be percent encoded to prevent a conflict with paths. Directories and subdirectories **MAY** optionally be included, delimited by colons rather than slashes.

As specified in the following Augmented Backus-Naur Form (ABNF) notation [\[RFC2234\]](#) the [SCID](#) **MUST** be present in the DID string. See examples below. The `domain-segment` and `path-segment` elements refer to [\[RFC3986\]](#)'s ABNF for a Generic URL (page 49). Attempting to replicate here the full ABNF of those elements from that RFC would inevitably be wrong.

```
webvh-did = "did:webvh:" scid ":" domain-segment 1+( "." domain-segment ) [ percent-en
scid = 46(base58-alphabet) ; The characters in the base58-btc-alphabet are as defined
domain-segment = ; A part of a domain name as defined in RFC3986, such as "example" an
percent-encoded-port = "%3A" ( "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ) 1
path-segment= ; A part of a URL path as defined in RFC3986, such as "path", "to", "fol
```

The ABNF for a `did:webvh` is almost identical to that of `did:web`, with changes only to the DID Method (`webvh` instead of `web`), and the addition of the `<scid>`: (defined in the [SCID](#)) section of this specification) element in `did:webvh` that is not in `did:web`. As specified in the [DID-to-HTTPS Transformation](#) section of this specification, `did:webvh` and `did:web` DIDs that have the same fully qualified domain and path transform to the same HTTPS URL, with the exception of the final file – `did.json` for `did:web` and `did.jsonl` for `did:webvh`. For `did:webvh` DIDs using [witnesses](#), another file `did-witness.json` is also found (logically) beside the `did.jsonl` web server file. See the [witnesses](#) section of this specification for details.

§ 3.4 The DID to HTTPS Transformation

The `did:webvh` [method-specific identifier](#) is defined to enable a transformation of the DID to an HTTPS URL for publishing and retrieving the [DID Log](#). This section defines the transformation from DID to HTTPS URL, including a number of examples.

Given a `did:webvh`, the HTTPS URL for the [DID Log](#) is generated by carrying out the following steps. The steps are carried out by the [DID](#) to determine where to publish the [DID Log](#), and by all resolvers to retrieve the [DID Log](#). The process described here includes the appropriate handling of [international domain names](#).

1. **Remove the 'did:webvh:' prefix** from the input identifier.
2. **Remove the SCID segment**, which is the first segment after the prefix.
3. **Transform the domain segment**, the first segment (up to the first `:` character) of the remaining string.
 - If the domain segment contains a port, decode percent-encoding and preserve the port.
 - Apply Unicode normalisation as defined in [\[RFC3491\]](#) (see this explainer on [Unicode normalization](#)).
 - Apply IDNA (Punycode) encoding as per IDNA2008 [\[RFC9233\]](#). See the [FAQ on IDNA](#) for more details. For domains that do not contain international domain name elements, this should result in no change.
4. **Transform the path**, the 0 or more segments after the first `:` character, delimited by `:` characters.
 - Percent-encode each path segment per [\[RFC3986\]](#) to ensure a valid HTTP URL.
 - Replace each `:` character separator between the segments with a `/` character to create the encoded path (`{encoded_path}`).
5. **Reconstruct the HTTPS URL:**
 - Format as `https://{domain}:{port}/{encoded_path}/did.jsonl` if a port is present.
 - Format as `https://{domain}/{encoded_path}/did.jsonl` if there are path segments and no port.
 - If no path segments exist, format as `https://{domain}:{port}/.well-known/did.jsonl` or `https://{domain}/.well-known/did.jsonl` as applicable.
6. The content type for the `did.jsonl` file **SHOULD** be `text/jsonl`.

If the DID is using [witnesses](#), an extra JSON file containing the witness proofs for the [DID Log Entries](#) must be published and retrieved during resolution. The URL for the extra file is defined by replacing the `/did.jsonl` at the end of the [DID Log](#) URL with `/did-witness.json`.

When this algorithm is used for resolving a DID path (such as `<did>/whois` or `<did>/path/to/file` as defined in the section [DID URL Handling](#)) using the implicit

`services` , update step 5. to not include the `.well-known/` path segment, and to append the DID URL path instead of `did.jsonl` .

The following are some examples of various DID-to-HTTPS transformations based on the processing steps specified above.

EXAMPLE

`did:webvh` DIDs and the corresponding web locations of their `did:webvh` log file. In the examples, `{SCID}` is a placeholder for where the generated `SCID` will be placed in the actual DIDs and HTTPS URLs. Note that when the `{SCID}` follows the literal `did:webvh:` as a separate element, the `{SCID}` is not part of the HTTPS URL.

domain/ `did:web-compatible`

`did:webvh:{SCID}:example.com -->`

`https://example.com/.well-known/did.jsonl`

subdomain

`did:webvh:{SCID}:issuer.example.com -->`

`https://issuer.example.com/.well-known/did.jsonl`

path

`did:webvh:{SCID}:example.com:dids:issuer -->`

`https://example.com/dids/issuer/did.jsonl`

path w/ port

`did:webvh:{SCID}:example.com%3A3000:dids:issuer -->`

`https://example.com:3000/dids/issuer/did.jsonl`

internationalized domain

`did:webvh:{SCID}:jp . .jp: -->`

`https://xn--jp-cd2fp15c.xn--fsq.jp/%E7%94%A8%E6%88%B7/did.jsonl`

A client resolving a `did:webvh` DID **MAY** choose to use a [watcher](#) as the source of data about a `did:webvh` DID, rather than resolving the DID's HTTPS location to retrieve the [DID Log](#). See the specification section on [Watchers](#) for information about `did:webvh` and [watchers](#).

The location of the `did:webvh did.jsonl` [DID Log](#) file is the same as where the comparable `did:web`'s `did.json` file is published. A [DID](#) **MAY** publish both DIDs and so, both files. The process to do so is described in the [publishing a parallel did:web DID](#) section of this specification.

WARNING

While the transformation from a `did:webvh` identifier to an HTTPS resource relies on DNS resolution, clients should not assume that a `did:webvh` identifier is inherently bound to or controlled by the entity associated with the corresponding DNS domain. In fact, a `did:webvh` [DID Log](#) may be obtained from sources other than its corresponding HTTPS location (perhaps indexed by its [SCID](#)), and in such cases, the same verification steps may be applied to determine its validity.

Verification of a `did:webvh` identifier using this specification ensures cryptographic validity, but that does not imbue “trust” in the identifier itself. Trust in a `did:webvh` DID should be derived from external sources, such as verifiable credentials issued by trusted parties (possibly discovered by resolving the DID’s [/whois](#) URL) or via Trust Registries that maintain authoritative records of trusted DIDs in a given context. Implementers should exercise caution and avoid conflating technical verification with trustworthiness, ensuring that reliance on a `did:webvh` identifier is informed by independent verification mechanisms.

§ 3.5 The DID Log File

The [DID log](#) file contains a list of [entries](#), one for each version of the DID. A version of the DID is an update to the contents of the resolved [DIDDoc](#) for the DID, and/or a change to the [parameters](#) that control the generation and verification of the DID.

Each entry is a JSON object consisting of the following properties.

```
{ "versionId": "", "versionTime": "", "parameters": {}, "state": {}, "proof" : [] }
```

1. The [\[JSON-SCHEMA-CORE\]](#) definition of the [DID log entry](#) data structure can be found in the [log_entry.json](#) file in this repository.
2. The value of `versionId` **MUST** be a string consisting of the DID version number (starting at `1` and incrementing by one per DID version), a literal dash `-`, and the `entryHash`, a hash calculated across the [log entry](#) content. The input to the hash is chosen so as to link each entry to its predecessor in a ledger-like chain. The input to the hash is specified in the [Entry Hash Generation and Verification](#) section of this specification.
3. The value of `versionTime` **MUST** be a timestamp in UTC of the entry in [ISO8601](#) format, as asserted by the [DID Controller](#). The timestamp **MUST** be the time the DID will be retrieved by a [witness](#) or resolver, or before.
4. The JSON object `parameters` contains the configurations/options set by the [DID Controller](#) to be used in the processing of current and future [log entries](#). Permitted `parameters` are defined in the `did:webvh` [DID Method Parameters](#) section of this specification.
5. The JSON object `state` contains the [DIDDoc](#) for this version of the DID.

6. The JSON array **proof** contains a [Data Integrity](#) proof created for the entry and signed by a key authorized to update the [DIDDoc](#).

After creation, each entry has (per the [JSON Lines](#) specification) all extra whitespace removed, a `\n` character appended, and the result added to the [DID Log](#) file for publication.

A more comprehensive description of how to create and update a [DID log](#) is given in steps 4 - 6 of the [create DID](#) section.

Examples of [DID Logs](#) and [DID log entries](#) can be found in the [Examples](#) section on the [did:webvh](#) information website.

§ 3.6 DID Method Operations

§ 3.6.1 Create (Register)

Creating a [did:webvh](#) DID is done by carrying out the following steps.

1. **Define the DID string** The start of the DID **MUST** be the literal string “[did:webvh:](#)[{SCID}](#)”, where the [{SCID}](#) is a placeholder that will be replaced by the calculated [SCID](#) later in the process (see step 5). This first part of the DID string is followed by a fully qualified domain name (with an optional path) that is secured by a TLS/SSL certificate and reflects the web location at which the [DID Log](#) ([did.jsonl](#)) will be published

The DID **MUST** be a valid [did:webvh](#) DID as per the ABNF of a [did:webvh](#) DID defined in the [Method-Specific Identifier](#) section of this specification.

1. Note: the [SCID](#) for a [did:webvh](#) DID is not by default in the HTTPS URL for the DID. A [DID Controller](#) **MAY** include the [SCID](#) in the HTTPS URL by inserting additional placeholder [{SCID}](#) strings into the domain name or path components of the method-specific identifier when creating the DID. Additional instance(s) of the [SCID](#) in the domain and/or path parts of the DID does not alter the [DID-to-HTTPS transformation](#).
2. **Generate the authorization key pair(s)** [Authorized keys](#) are authorized to control (create, update, deactivate) the DID. At the same time, generate any other key pairs that will be placed into the initial [DIDDoc](#) for the DID.
 1. If the DID is to use [pre-rotation](#), additional key generation will be necessary to generate the required “next” authorization keys and their corresponding [pre-rotation](#) hashes.
 2. For each authorization key pair, generate a [multikey](#) based on the key pair’s public key. The [multikey](#) representations of the public keys are placed in the [updateKeys](#) property in [parameters](#).
 3. The public key(s) of the authorization key pair(s) **MAY** be used in the [DIDDoc](#) as well, but that is not required.

3. **Create the initial [DIDDoc](#) for the DID** The [DIDDoc](#) **MUST** contain the top level `id` property which **MUST** be the DID string from step 1, including the placement of the `{SCID}` placeholder for the [SCID](#). Other [DIDDoc](#) verifications **SHOULD** be performed.

All other absolute reference's to the DID in the [DIDDoc](#) must use the form defined in step 1, with the identified placeholder for the [SCID](#) (e.g., `did:webvh:{SCID}:example.com#key-1`, `did:webvh:{SCID}:example.com:dids:issuer#key-1`, etc.).

The [DIDDoc](#) can contain any other content as deemed necessary by the [DID Controller](#).

1. Note: The placeholder (the string `{SCID}`) **MUST** be in every place in the [DIDDoc](#) where the [SCID](#) is to be placed.
4. **Generate a preliminary DID Log Entry JSON object** containing the same JSON properties that will be in the published [DID log entry](#), but with some values preset, pending calculation of the [SCID](#) and [entryHash](#) and without the `proof`.
 1. The value of `versionId` string **MUST** be the placeholder literal `"{SCID}"`.
 2. The value of `versionTime` string **MUST** be a valid UTC [ISO8601](#) date/time string, and the represented time **MUST** be before or equal to the current time.
 3. The value of the `parameters` property **MUST** be a JSON object defined at the discretion of the [DID Controller](#). The properties in this nested JSON object **MUST** be as permitted in the [DID Generation and Verification Parameters](#) section of this specification, and all required values in the first version of the DID **MUST** be present. In addition, where the [SCID](#) of the DID is referenced in the parameters, the placeholder literal string `{SCID}` **MUST** be used in place of the to-be-calculated [SCID](#).
 4. The value of the `state` property **MUST** be the initial [DIDDoc](#) as defined in the previous step 3 of this process.
5. **Update the preliminary DID Log Entry to the initial DID Log Entry** Use the preliminary [DID log entry](#) to perform the consecutive steps:
 1. **Calculate the [SCID](#)** The preliminary JSON object **MUST** be used to calculate the [SCID](#) for the DID as defined in the [SCID Generation and Verification](#) section of this specification.
 2. **Replace the placeholder `{SCID}`** Replace throughout the preliminary JSON object the placeholder `"{SCID}"` with the calculated [SCID](#) from the previous step.
 3. **Calculate the [Entry Hash](#)** The preliminary JSON object updated in the previous step **MUST** be used to calculate the [Entry Hash](#) (`entryHash`) for the [log entry](#), as defined in the [Entry Hash Generation and Verification](#) section of this specification.

4. **Replace the preliminary `versionId` value** The value of the `versionId` property **MUST** be updated with the literal string `1` (for version number 1), a literal `-`, followed by the `entryHash` value calculated in the previous step.
5. **Generate the [Data Integrity](#) proof** A [Data Integrity](#) proof on the preliminary JSON object as updated in the previous step **MUST** be generated using an authorized key in the required `updateKeys` property in the [parameters](#) object and the `proofPurpose` set to `assertionMethod`.
6. **Add the [Data Integrity](#) proof** The [Data Integrity](#) proof is added to the preliminary JSON object. The resultant JSON object is the initial [DID log entry](#) for the DID.
6. **Generate the first [JSON Line](#)** The [DID log entry](#) **MUST** be updated to be a [JSON Lines](#) entry by removing extraneous white space and appending a carriage return, and the result stored as the contents of the file `did.jsonl`.

If the [DID Controller](#) has opted to use [witnesses](#) for the DID, the required proofs from the DID's [witnesses](#) **MUST** be collected and published in the `did-witness.json` file before the [DID](#) with the new version is published. See the [DID Witnesses](#) section of this specification.

7. **Publish the [DID Log](#)** The complete [DID Log](#) file **MUST** be published at the appropriate Web location defined by the `did:webvh` DID identifier (see step 1)
 - This is a logical operation – how a deployment serves the `did.jsonl` content is not constrained.
 - Use the [DID-to-HTTPS Transformation](#) steps to transform the DID into the Web location of the [DID Log](#) file.

If there are [watchers](#) configured for the DID, a webhook is triggered to notify the [watchers](#) that a new DID is available and should be retrieved. See the [Watchers](#) section of this specification for more details.

A controller **MAY** generate an equivalent `did:web` [DIDDoc](#) and publish it as defined in the [Publishing a Parallel `did:web` DID](#) section of this specification. The `did:web` [DIDDoc](#) could be used for backwards compatibility as a transition is made from `did:web` to `did:webvh`. Verifiers using the `did:web` lose the verifiable properties and history of the `did:webvh` for the convenience of the simple retrieval of the `did:web` [DIDDoc](#).

§ 3.6.2 Read (Resolve)

The following steps **MUST** be executed to resolve the [DIDDoc](#) for a `did:webvh` DID:

1. The [DID-to-HTTPS Transformation](#) steps **MUST** be used to transform the DID into an HTTPS URL for the [DID](#) file.

2. Perform an HTTPS **GET** request to the URL using an agent that can successfully negotiate a secure HTTPS connection, which enforces the security requirements as described in [Security and privacy considerations](#).
3. When performing the DNS resolution during the HTTPS GET request, the client SHOULD utilize [\[RFC8484\]](#) in order to prevent tracking of the identity being resolved.
4. The [DID Log](#) file **MUST** be processed as described below.

To process the retrieved [DID Log](#) file, the resolver **MUST** carry out the following steps on each of the [log entries](#) in the order they appear in the file, applying the [parameters](#) set from the current and previous entries. As noted in the [DID Log File](#) section, [log entries](#) are each a JSON object with the following properties:

1. **versionId**
2. **versionTime**
3. **parameters**
4. **state** – the version's [DIDDoc](#).
5. **proof** – a [Data Integrity](#) proof for the [log entry](#).

For each entry:

1. Update the currently active [parameters](#) with the [parameters](#) from the entry (if any). The **parameters** **MUST** adhere to the [did:webvh DID Method Parameters](#) section of this specification. Continue processing using the now active set of [parameters](#).
 - While all [parameters](#) in the first [Log Entry](#) take effect immediately, some kinds of [parameters](#) defined in later [entries](#) only take effect *after* that entry has been published. For example, updating the **nextKeys** and **witnesses** arrays take effect only *after* the entry in which they are defined has been published.
2. The [Data Integrity](#) proof in the entry **MUST** be valid and signed by an authorized key as defined in the [Authorized Keys](#) section of this specification, and with a **proofPurpose** set to **assertionMethod**.
 1. If the [DID Controller](#) has opted to use [witnesses](#) resolvers **MUST** retrieve and verify the DID's **did-witness.json** file. For details, see the [DID Witnesses](#) section of this specification.
3. Verify the **versionId** for the entry. The **versionId** is the concatenation of the version number, a dash (-), and the **entryHash**.
 1. The version number **MUST** be **1** for the the first [log entry](#) and **MUST** be incremented by one for each subsequent [log entry](#).
 2. A dash – **MUST** follow the version number.

3. The **entryHash** **MUST** follow the dash, and **MUST** be verified using the process defined in the [Entry Hash Generation and Verification](#) section of this specification.
4. The **versionTime** **MUST** be a valid UTC [ISO8601](#) date/time string. The **versionTime** for each [log entry](#) **MUST** be greater than the previous entry's time. The **versionTime** of the last entry **MUST** be earlier than the current time.
5. When processing the first [DID log](#) entry, verify the [SCID](#) (defined in the [parameters](#)) according to the [SCID Generation and Verification](#) section of this specification.
6. Get the value of the [log entry](#) property **state**, which is the [DIDDoc](#) for the version.
 1. Track and check that the DID being resolved matches the top-level **id** in at least one version of the [DIDDoc](#); otherwise, the resolution **MUST** be flagged as invalid.
7. If [Key Pre-Rotation](#) is being used, the hash of all **updateKeys** entries in the **parameters** property **MUST** match a hash in the array of **nextKeyHashes** [parameter](#) from the previous [DID log](#) entry, with exception of the first entry, as defined in the [Pre-Rotation](#)[[Key Pre-Rotation Hash Generation and Verification](#)](#pre-rotation-key-hash-generation-and-verification) section of this specification.
8. As each [log entry](#) is processed and verified, collect the following information about each version:
 1. The [DIDDoc](#).
 2. The **versionId** of the [DIDDoc](#).
 3. The UTC **versionTime** of the [DIDDoc](#).
 4. The latest list of active [multikey](#) formatted public keys authorized to update the DID, from the **updateKeys** lists in the [parameters](#).
 5. If [pre-rotation](#) is being used, the hashes of authorized keys that must be used in the **updateKeys** list of the next [DID log](#) entry. The [pre-rotation](#) hashes are in the **nextKeyHashes** list in the [parameters](#).
 6. All other **did:webvh** processing configuration settings as defined by in the **parameters** object.
 7. Add the value of top level **id**
9. If the **parameters** for any of the versions define that some or all of the [DID Log entries](#) must be witnessed, further verification of the [witness](#) proofs must be carried out, as defined in the [DID Witnesses](#) section of this specification.
10. Flag failed verifications appropriately, either invalidating the entire DID or marking all entries from the first invalid entry to the end of the log as invalid.
11. Respond to the resolution request based on verification results:

1. If all verifications pass, resolve the DID, applying any query parameters as requested.
2. If the request includes query parameters (e.g., `?versionId=` or `?versionTime=`) that reference valid [DID log entries](#), return the corresponding [DIDDoc](#) version with a successful status code—even if later entries in the log are invalid.
3. If the DID or DID version being resolved is invalid, return an appropriate error code.

While resolver caching policies are an implementation matter and largely outside the scope of this specification, resolvers **SHOULD NOT** cache a DID that fails verification. This ensures that the DID's [DID Controller](#) has the opportunity to recover a DID that may have been erroneously or maliciously invalidated.

A resolver **MAY** use a DID [watcher](#) in addition to, or in place of retrieving the DID information from the source, and use that information based on their knowledge of the governance of the [watcher](#). See the [Watchers](#) section of this specification for more details.

As defined in the `[[spec:DID-RESOLUTION]]` specification, a `did:webvh` resolver should return the following DID Document Metadata when resolving a `did:webvh` DID Document:

```
{
  "versionId": "1-QmRRaLXwc6BjBuBPosSupJwEQ8w9f3znP7yfbpGfwcnLr6",
  "versionTime": "2025-01-23T04:12:36Z",
  "created": "2025-01-23T04:12:36Z",
  "updated": "2025-01-23T04:12:36Z",
  "scid": "QmPEQVM1JPTyrvEgBcDXwjK4TeyLGSX1PxjgyeAisdWM1p",
  "portable": false,
  "deactivated": false,
  "ttl": "3600",
  "witness": { ...
},
  "watchers": [ ...
]
```

where the items in the Metadata JSON object are:

- `versionId` — The `versionId` from the [Log Entry](#) of the resolved DIDDoc version.
- `versionTime` — The `versionTime` from the [Log Entry](#) of the resolved DIDDoc version, in [ISO8601](#) timestamp format.
- `created` — The [ISO8601](#) timestamp of the DID's first [log entry](#), indicating when (according to the [DID Controller](#)) the DID was created.
- `updated` — The [ISO8601](#) timestamp of the DID's last valid [log entry](#).
- `scid` — The [SCID](#) of the resolved DID.

- **portable** — A boolean value indicating whether the resolved DID has [portability](#) active and so may be moved in the future, as defined in the [portability](#) section of this specification.
- **deactivated** — A boolean indicating whether the DID has been deactivated. When **true**, the DID is no longer active.
- **ttl** - A string containing the unsigned integer value of the DID's **ttl** [\[\[ref; parameter\]\]](#) (time-to-live) in seconds. The TTL is guidance from the [DID Controller](#) for those resolving the DID about how long to cache the DID. The value is a string containing the integer value because the [\[\[spec: DID-RESOLUTION\]\]](#) specification requires that DID metadata not be integers. The value needs to be converted to an integer by the resolver client before use.
- **witness** — An object containing the current (active in the last valid [DID log entry](#)) configuration of witnesses for the DID. The object is as defined as the same named object in the [witness list](#) section of this specification.
 - The value of the **threshold** attribute of the **witness** object is a string containing the integer value of **threshold** because the [\[\[spec: DID-RESOLUTION\]\]](#) specification requires that DID metadata not include integers. The value needs to be converted to an integer by the resolver client before use.
- **watchers** — An array containing the current (active in the last valid [DID log entry](#)) list of [watcher](#) URLs that have agreed to monitor and cache the DID's state.

The “last valid [log entry](#)” for some of the items above references the case where a DID resolution request references a DIDDoc that was valid, but where the DID Log has later [log entries](#) that fail verification, as noted in the DID resolution steps earlier in this section. If all [log entries](#) pass verification, the last valid [log entry](#) is the last [log entry](#).

When a DID resolution error occurs, the **error** field **MUST** be included in the **didResolutionMetadata**, as defined by the [\[\[spec:DID-RESOLUTION\]\]](#) specification. In addition, resolvers **SHOULD** provide supplemental “Problem Details” metadata following [\[RFC9457\]](#), using the following structure:

```
"didResolutionMetadata": {
  "error": "invalidDid",
  "problemDetails": {
    "type": "https://w3id.org/security#INVALID_CONTROLLED_IDENTIFIER_DOCUMENT_ID",
    "title": "The resolved DID is invalid.",
    "detail": "Parse error of the resolved DID at character 3, expected ':'."
  }
}
```

As described in [\[\[spec:DID-EXTENSION-RESOLUTION\]\]](#), the following values **MUST** be used in the **error** field of the resolution metadata when resolving a **did:webvh** DID under the corresponding error conditions:

- **notFound** — The [DID Log](#) or the resource referenced by a DID URL was not found. If the [DID Log](#) does not exist at the DID's designated HTTPS location (according to the [DID-to-HTTPS Transformation](#)), the resolver **MAY** attempt to retrieve it from alternative sources, such as [Watchers](#), for verification and resolution.
- **invalidDid** — Any error that renders the **did:webvh** DID invalid during resolution.

Resolvers **SHOULD** populate the **problemDetails** field to aid in diagnosing and understanding resolution failures. The [did:webvh information site](#) may serve as a non-normative reference for common **did:webvh** resolution error types and explanations.

§ Reading did:webvh DID URLs

A **did:webvh** resolver **MUST** resolve the [\[DID-CORE\]](#) **versionId** and **versionTime** DID URL query parameters. The **versionId** query argument value **MUST** match the full **versionId** from a [DID Log entry](#) for the resolver to return that version of the DIDDoc. If a [DID Log entry](#) with that **versionId** is not found, a **NotFound** **MUST** be returned. A specified time in [ISO8601](#) format as the query argument for **versionTime** **MUST** return the DIDDoc from the [DID Log entry](#) that was active at that time, if any. If the DID was not active at the specified time, a **NotFound** **MUST** be returned.

A **did:webvh** resolver **SHOULD** resolve the DID URL query parameter **versionNumber** with an integer value if there is a [DID Log entry](#) with a **versionId** with a matching integer prior to the literal **-** — the **versionNumber** for that [DID Log entry](#) as defined in the process for setting the **versionId** in the [creating the DID](#) section of this specification. The **versionNumber** query parameter is not in the [\[DID-CORE\]](#) specification.

A **did:webvh** resolver **MAY** implement the resolution of the **/whois** and a DID URL Path using the [whois LinkedVP Service](#) and [DID URL Path Resolution Service](#) as defined in this specification by processing the [DID Log](#) and then dereferencing the DID URL based on the contents of the [DIDDoc](#). The client of a resolver that does not implement those capabilities must use the resolver to resolve the appropriate [DIDDoc](#), and then process the resulting DID URLs themselves. Since the default DID-to-HTTPS URL transformation is trivial, **did:webvh** [DID Controllers](#) are strongly encouraged to use the default behavior for DID URL Path resolution.

§ 3.6.3 Update (Rotate)

To update a DID, a new, verifiable [DID Log Entry](#) must be generated, witnessed (if necessary), appended to the existing [DID Log](#) (**did.jsonl**), and published to the web location defined by the DID. The process to generate a verifiable [DID Log Entry](#) follows a similar process to the [Create](#) process, as follows:

1. Make the desired changes to the [DIDDoc](#). The top-level **id** in the [DIDDoc](#) **MUST** contain the value of the DID.

1. If the DID is configured to support [portability](#), the root `id` property in the [DIDDoc](#) **MAY** be changed when the [DID Controller](#) wants to (or is forced to) publish the DID at a different Internet location and wants to retain the [SCID](#) and history of the DID. For details, see the [DID Portability](#) section of this specification.
2. Define the [parameters](#) JSON object to include the properties that affect the evolution of the DID. The `parameters` **MUST** be from those listed in the [did:webvh DID Method Parameters](#) section of this specification. Any [parameters](#) defined in the JSON object override the previously active value, while any [parameters](#) not included imply the existing values remain in effect. If no changes to the [parameters](#) are needed, an empty JSON object `{}` **MUST** be used.
 - While all [parameters](#) in the first [Log Entry](#) take effect immediately, some types of [parameters](#) defined in later [entries](#) only take effect after the entry has been published. For example, rotating the keys authorized to update a DID or changing the [witnesses](#) for a DID take effect only *after* the entry in which they are defined has been published.
3. Generate a preliminary [DID log entry](#) JSON object containing the following properties:
 1. The value of `versionId` **MUST** be the value of `versionId` from the *previous* [DID log entry](#).
 2. The `versionTime` value **MUST** be a string that is an [ISO8601](#) format UTC timestamp. The time **MUST** be greater than the time of the previous [log entry](#), and **MUST** be the time the DID will be retrieved by a [witness](#) or resolver, or before.
 3. The [parameters](#) passed in as a JSON object.
 4. Set the `state` JSON object to be the new version of the [DIDDoc](#).
4. Calculate the new `versionId` of the new [DID Log Entry](#), including incrementing the version number integer and using the process described in the [Entry Hash Generation and Verification](#) section of this specification.
5. Replace the value of the `versionId` property in the preliminary [DID Log](#) with the value produced in the previous step.
6. Generate a [Data Integrity](#) proof on the [DID log entry](#) using an authorized key, as defined in the [Authorized Keys](#) section of this specification, and the `proofPurpose` set to `assertionMethod`.
7. If [Key Pre-Rotation](#) is being used, the hash of all `updateKeys` entries in the `parameters` property **MUST** match a hash in the array of `nextKeyHashes` `parameter` from the previous [DID log](#) entry with exception of the first entry, as defined in the [Pre-Rotation](#)[[Key Pre-Rotation Hash Generation and Verification](#)](#pre-rotation-key-hash-generation-and-verification) section of this specification.
8. The proof JSON object **MUST** be added as the value of the `proof` property in the [log entry](#).
9. The entry **MUST** be made a [JSON Line](#) by removing extra whitespace, adding a `\n` to the entry.

10. If the [DID Controller](#) opts to use [witnesses](#) for the DID, the [DID Controller](#) **MUST** collect the [threshold](#) of proofs from the DID's [witnesses](#), and update and publish the DID's `did-witness.json` file. The updated `did-witness.json` file **MUST** be published **BEFORE** the updated [DID Log](#) file is published. See the [DID Witnesses](#) section of this specification.
11. The new [log entry](#) **MUST** be appended to the existing contents of the [DID Log](#) file `did.jsonl`.
12. The updated [DID Log](#) file **MUST** be published the appropriate location defined by the `did:webvh` identifier.
 - This is a logical operation – how a deployment serves the `did.jsonl` content is not constrained.
 - If there are [watchers](#) configured for the DID, webhooks are triggered to notify the [watchers](#) that an update is available and should be retrieved. See the [Watchers](#) section of this specification for more details.

A controller **MAY** generate an equivalent, updated `did:web DIDDoc` and publish it as defined in the [Publishing a Parallel did:web DID](#) section of this specification.

§ 3.6.4 Deactivate (Revoke)

Deactivating a DID allows a [DID Controller](#) to signal that the DID is no longer being maintained or updated. There is an explicit approach to deactivation that aligns with the [\[DID-CORE\]](#) specification, and other approaches that a `did:webvh DID Controller` can use if the [\[DID-CORE\]](#) approach does not achieve their desired outcome. This section covers the various methods for signaling that a DID is retired.

To deactivate a `did:webvh` DID per the [DID-CORE](#) specification, the [DID Controller](#) **MUST** add to the [DID log entry parameters](#) the property name and value `"deactivated": true`. Once done, a resolver **MUST NOT** return the [DIDDoc](#) and **MUST** include `"deactivated": true` in the DID Resolution Metadata. A [DID Controller](#) deactivating a `did:webvh` DID **MAY** update some [parameters](#) attributes to further indicate the deactivation of the DID, such as setting the `updateKeys` array to `[]`, preventing further versions of the DID. If the DID is using [pre-rotation](#), two [DID log entries](#) are required to accomplish that state: the first to stop the use of pre-rotation, and the second to set `updateKeys` to `[]`. For additional details about turning off [pre-rotation](#) see the [pre-rotation](#) section of this specification.

A concern with using the [\[DID-CORE\]](#) approach to deactivation is the resolver requirement that the [DIDDoc](#) not be returned for a deactivated DID. A [DID Controller](#) might want the final [DIDDoc](#) to continue to be resolved, while also signaling that the DID is no longer being updated. In the future, a DID Resolution query parameter (`returnDeactivatedDidDocument=true`) has been proposed to be added to the [\[\[spec: DID-RESOLUTION\]\]](#) specification to allow a client to request the [DIDDoc](#) of a deactivated DID. However, even if that parameter is adopted, it places the burden on the resolver client to decide whether and how to use it. An alternative that a `did:webvh DID Controller` could use

is to signal that a DID is no longer being updated by setting the `updateKeys` array to empty (`[]`), as discussed above, and not setting the `deactivated` `parameter` to `true` . The result is that the final `DIDDoc` continues to be returned by default, and the DID Resolution Metadata indicates that the DID can no longer be updated.

To resolve a prior version of a deactivated `did:webvh` DID, a client can use the appropriate DID Resolution query `parameters` `versionId` , `versionTime` , or the `did:webvh`-specific `versionNumber` (as described in the Read (Resolve) section of this specification). When such a DID is resolved in this way, the DID Resolution Metadata **MUST** include the property name and value `"deactivated": true` .

A `DID Controller` can “deactivate” a DID by removing the published `DID Log` and associated files and resources. Once removed, attempts to retrieve the `DID Log` will result in an `Not Found` error status when resolving the DID. `Watchers` monitoring a removed DID **SHOULD** continue to cache the last known valid state of the DID indefinitely so that their clients can still resolve and reference it, even after the `DID Log` has been deleted.

§ 3.7 DID Method Processes

The `DID Method Operations` reference several processes that are executed during `DIDDoc` generation and DID resolution verification. Each of those processes is specified in the following sections.

§ 3.7.1 `did:webvh` DID Method Parameters

All `did:webvh` `Log entries` contain the JSON object `parameters` . This object defines the DID processing `parameters` used by the `DID Controller` when publishing the current and subsequent `DID log entries`. DID Resolvers **MUST** use the same `parameters` to process the `DID Log` to resolve the DID. The `parameters` object **MUST** only include properties defined in the version of the `did:webvh` DID Method specification being used.

General Rules for Parameters:

- **Default Values:** When the `method` parameter (see below) sets the `[SEMVER]` version of this specification to be used for a DID, any parameter introduced by that version but not explicitly set in the same `log entry` **MUST** assume the default value defined in this section. The `method` parameter is required in the first `log entry` and may appear in later entries to upgrade the DID to a newer `[SEMVER]` version of the `did:webvh` specification.
- **Allowed Values:** Each parameter is constrained by the data type, structure and allowed values specified in this section. If a value does not conform, the parameter is invalid and resolvers **MUST** reject the `log entry`.
- **Deactivation:** Parameters that support deactivation (such as `witness` or `nextKeyHashes`) are set to defined values, described below, to indicate they are no

longer active.

- The JSON `null` value **MUST NOT** be used to indicate default or deactivated values, as it removes the typing information required for proper interpretation.

NOTE

Some early `did:webvh` implementations used the JSON `null` value to indicate the deactivation of parameters such as `watchers`, `witness`, `updateKeys`, `nextKeyHashes`, and `ttl`. Although this usage is deprecated and not valid per the current specification, resolver implementations **SHOULD** gracefully accept `null` and immediately convert the value to their equivalent default value for the parameter when processing [DID Log entries](#).

EXAMPLE

An example of the `parameters` property in the first [DID Log entry](#):

```
{
  "portable": true,
  "updateKeys": [
    "z82LkqR25TU88tztBEiFydNf4fUPn8oWBANckcmuqgonz9TAbK9a7WGQ5dm7jyqyRMpaRAe"
  ],
  "nextKeyHashes": [
    "enkkrohe5ccxyc7zghic6qux5inyzthg2tqka4b57kvtorysc3aa"
  ],
  "method": "did:webvh:1.0",
  "scid": "{SCID}"
}
```

The following lists the [parameters](#), their data types, and enumerated values.

- **method**: Specifies the `did:webvh` [\[SEMVER\]](#) specification version to be used for processing the DID's log. Each acceptable value in turn defines what cryptographic algorithms are permitted for the current and subsequent [DID log entries](#). An update to the specification version in the middle of a [DID Log](#) could introduce new [parameters](#).
 - **MUST** appear in the first [DID log entry](#).
 - If not present in later [DID log entries](#), the previous value continues to apply.
 - **MAY** appear in later entries to change the DID processing rules to that of a new version of the specification.
 - **MUST** be set to a specification [\[SEMVER\]](#) version string equal to or higher than the currently active `method` setting.
 - Acceptable values:
 - `did:webvh:1.0`
 - Permitted hash algorithms: `SHA-256` [\[RFC6234\]](#)

- Permitted [Data Integrity](#) cryptosuites: [eddsa-jcs-2022](#) [[DI-EDDSA-V1.0](#)]
- **scid** : The [SCID](#) value for the DID.
 - **MUST** appear in the first [log_entry](#).
 - **MUST NOT** appear in later [log_entries](#).
- **updateKeys** : A JSON array of [multikey](#) formatted public keys associated with the private keys that are authorized to sign the log entries that update the DID. See the [Authorized Keys](#) section of this specification for additional details.
 - This property **MUST** appear in the first [log_entry](#) and **MAY** appear in subsequent entries.
 - If not present in later [DID log_entries](#), the previous value continues to apply.
 - A key from the active **updateKeys** array **MUST** be used to authorize the each [log entry](#), where active is defined as follows.
 - In the first [log_entry](#), the active **updateKeys** is the one defined in that entry.
 - In all other [log_entries](#) *without* [Key Pre-Rotation](#) active, the active **updateKeys** is that of the most recent **prior** [log_entry](#).
 - In all other [log_entries](#) *with* [Key Pre-Rotation](#) active, the active **updateKeys** is that of the most current [log_entry](#).
 - **updateKeys** **SHOULD** be set to an empty array `[]` when deactivating the DID. See the [deactivate](#) section of this specification for more details.
- **nextKeyHashes** : A JSON array of strings that are hashes of [multikey](#) formatted public keys that **MAY** be added to the **updateKeys** list in the next [log_entry](#). At least one entry of **nextKeyHashes** **MUST** be added to the next **updateKeys** list.
 - The process for generating the hashes and additional details for using [pre-rotation](#) are



Issues (10)

- If not set in the first [log_entry](#), its value defaults to an empty array (`[]`).
- If not set in other [log_entries](#), its value is retained from the most recent prior value.
- Once the **nextKeyHashes** parameter has been set to a non-empty array, [Key Pre-Rotation](#) is active. While active, the properties **nextKeyHashes** and **updateKeys** **MUST** be present in all [log_entries](#).
- While [Key Pre-Rotation](#) is active, all [multikey](#) formatted public keys added in a new **updateKeys** list **MUST** have their hashes listed in the **nextKeyHashes** list from the previous [log_entry](#).
- A [DID Controller](#) **MAY** include extra hashes in the **nextKeyHashes** array that are not subsequently used in an **updateKeys** entry. Any unused hashes in **nextKeyHashes** arrays are ignored.
- The value of **nextKeyHashes** **MAY** be set to an empty array (`[]`) to deactivate [pre-rotation](#). For additional details about turning off [pre-rotation](#), see the [Pre-Rotation Key](#)

[Hash Generation and Verification](#) section of this specification.

- **witness** : A JSON object declaring the set of witnesses and threshold number of witness proofs required to update the DID. For details of this data and its usage in the DID update approval process, see the [DID Witnesses](#) section of this specification...
 - Defaults to `{}` if not set in the first [log entry](#).
 - If not set in other [log entries](#), its value is retained from the most recent prior value.
 - If the **witness** property is updated from `{}`, the change is immediately active, and the corresponding [log entry](#) **MUST** be [witnessed](#).
 - The **witness parameter** **MAY** be set to `{}` to indicate that witnesses are not (or no longer) being used. If witnesses are active when the **witness parameter** is set to `{}`, that [log entry](#) **MUST** be [witnessed](#).
- **watchers** : An optional entry whose value is a JSON array containing a list of URLs ([\[RFC9110\]](#)) that have notified the DID Controller that they are willing to watch the DID. See the [Watchers](#) section of this specification for more details.
 - Defaults to `[]` if not set in the first [log entry](#).
 - If not set in other [log entries](#), its value is retained from the most recent prior value.
 - **MAY** be set to an empty array `[]` to indicate that watchers are not (or no longer) being used.
- **portable** : Boolean (JSON `true` / `false`) indicating if the DID is portable, allowing a DID Controller to control if a DID can be moved, while retaining its [SCID](#) and verifiable history. See the [DID Portability](#) section of this specification for more details.
 - Can **ONLY** be set to `true` in the first entry.
 - Defaults to `false` if omitted in the first entry.
 - Retains value if omitted in later entries.
 - Once set to `false`, **MUST NOT** be changed to `true`.
- **deactivated** : A JSON boolean that indicates whether the DID has been deactivated. A deactivated DID is no longer subject to updates but remains resolvable. See the [deactivate \(revoke\)](#) section of this specification for more details.
 - Defaults to `false` if not set in the first [DID log entry](#).
 - If set to `true`, the DID is considered deactivated and no further updates to the DID are permitted.
- **ttl** : An unsigned integer that indicates how long, in seconds, a resolver should cache the resolved `did:webvh` DID before refreshing. It provides guidance from the [DID Controller](#) on cache duration, with a range of 0 to 2³¹[\[RFC2181\]](#). The parameter is analogous to the **TTL** parameter used in DNS [\[RFC2181\]](#). Caching a `did:webvh` can be valuable in places where the business rules require resolving a number of DID URLs for the same DID. For example, a client might want call the resolver to the current [DIDDoc](#), and then make repeated calls to get all of the

previous versions of the [DIDDoc](#). By caching the [DIDDoc](#) state, the resolver would not have to retrieve and process the [DID Log](#) on each call.

- Defaults to **3600** (1 hour) if not set in the first [DID log entry](#).
- If set to **0**, indicates that the DID should not be cached.

§ 3.7.2 Cryptographic Agility

The **did:webvh** DID method is designed to support cryptographic agility—the ability to adapt to evolving cryptographic algorithms and suites over time without breaking compatibility or requiring global coordination.

Cryptographic agility in **did:webvh** is achieved through the following mechanisms:

- **Self-describing cryptographic formats:** All cryptographically generated data (e.g., hashes and signatures) use formats that encode the algorithm used. Hashes use the [Multihash](#) format, and signatures use the [Data Integrity](#) Proofs. These formats allow verifiers to determine the algorithm from the data itself, enabling flexible and extensible support.
- **Specification versioning via the **method** parameter:** Each [DID Log Entry](#) may include a **method** parameter that specifies the version of the **did:webvh** specification being used for the current and subsequent [log entries](#). This parameter is required in the initial log entry and may be updated in later entries to adopt newer versions of the specification. This allows long-lived DIDs to transition to updated algorithm sets over time.
- **Version-specific algorithm policies:** Each version of the **did:webvh** specification defines the permitted cryptographic algorithms and suites for that version. This constrains what algorithms [DID Controllers](#) may use and limits the verification requirements placed on resolvers. For example, the v1.0 specification permits only one hash algorithm and one [Data Integrity](#) cryptosuite, while future versions may change or expand this set.
- **Response to cryptographic vulnerabilities:** If flaws are identified in a cryptographic algorithm permitted by a given version of the specification, a new version of the **did:webvh** specification will be released that removes or replaces the compromised algorithms. DID Controllers may then rotate to the newer version by updating the **method** parameter in a new log entry.

This design allows **did:webvh** to remain interoperable and verifiable across cryptographic eras while minimizing the burden on resolvers and preserving backward compatibility where safe to do so.

§ 3.7.3 SCID Generation and Verification

The [self-certifying identifier](#) or **SCID** is a required [parameter](#) in the first [DID log entry](#) and is the hash of the DID's inception event.

§ Generate SCID

To generate the [SCID](#) for a `did:webvh` DID, the DID Controller **MUST** execute the following function:

```
base58btc(multihash(JCS

```
(preliminary log entry with placeholders), <hash
algorithm>))
```


```

Where:

1. The `preliminary [[ref: log entry]] with placeholders` consists of the following pre-publication JSON object of what will become the first [log entry](#). The placeholder is the literal string "`{SCID}`".
 - The `versionId` entry, which **MUST** be `{SCID}`.
 - The `versionTime` entry, which **MUST** be a string that is the current time in UTC [ISO8601](#) format, e.g., "`2024-04-05T07:32:58Z`".
 - The complete `parameters` for the initial [log entry](#) as defined by the [DID Controller](#), with the placeholder wherever the [SCID](#) will eventually be placed.
 - The `state` JSON object with the value being the initial [DIDDoc](#) with placeholders (the literal string "`{SCID}`") wherever the [SCID](#) will eventually be placed in the [DIDDoc](#).
2. `JCS` is an implementation of the [JSON Canonicalization Scheme \[RFC8785\]](#). It outputs a canonicalized representation of its JSON input.
3. `multihash` is an implementation of the [multihash](#) specification. Its output is a hash of the input using the associated `<hash algorithm>`, prefixed with a hash algorithm identifier and the hash size.
4. `<hash algorithm>` is the hash algorithm used by the [DID Controller](#). The hash algorithm **MUST** be one listed in the [parameters](#) defined by the version of the `did:webvh` specification being used by the [DID Controller](#).
5. `base58btc` is an implementation of the [base58btc](#) function. Its output is the base58 encoded string of its input.

§ Verify SCID

To verify the [SCID](#) of a `did:webvh` DID being resolved, the resolver **MUST** execute the following process:

1. Extract the first [DID log entry](#) and use it for the rest of the steps in this process.
2. Extract the `scid` property value from the [parameters](#) in the [DID log entry](#).
3. Determine the hash algorithm used by the [DID Controller](#) from the [multihash](#) `scid` value.

- The hash algorithm **MUST** be one listed in the [parameters](#) defined by the version of the `did:webvh` specification being used by the [DID Controller](#) based on the `method parameters` property.
- 4. Remove the [data integrity](#) proof property from the [DID log entry](#).
- 5. Replace the `versionId` property value with the literal `"{SCID}"`.
- 6. Treat the resulting [log entry](#) as a string and do a text replacement of the `scid` value from Step 2 with the literal string `{SCID}`.
- 7. Use the result and the hash algorithm (from Step 3) as input to the function defined in the [Generate SCID](#) section (above).
- 8. The output string **MUST** match the `scid` extracted in Step 2. If not, terminate the resolution process with an error.

§ 3.7.4 Entry Hash Generation and Verification

The `entryHash` follows the version number and dash character `-` in the `versionId` property in each DID log entry. Each `entryHash` is calculated across its [log entry](#), excluding the [Data Integrity](#) proof. The `versionId` used in the input to the hash is a predecessor value to the current [log entry](#), ensuring that the [entries](#) are cryptographically “chained” together in a microledger. For the first [log entry](#), the predecessor `versionId` is the SCID (itself a hash), while for all other entries it is the `versionId` property from the previous log entry.

§ Generate Entry Hash

To generate the required hash for a `did:webvh` [log entry](#), the [DID Controller](#) **MUST** execute the process `base58btc(multihash(JCS(entry), <hash algorithm>))` given a preliminary [log entry](#) as the string `entry`, where:

1. `JCS` is an implementation of the [JSON Canonicalization Scheme](#) ([RFC8785]). Its output is a canonicalized representation of its input.
2. `multihash` is an implementation of the [multihash](#) specification. Its output is a hash of the input using the associated `<hash algorithm>`, prefixed with a hash algorithm identifier and the hash size.
3. `<hash algorithm>` is the hash algorithm used by the [DID Controller](#). The hash algorithm **MUST** be one listed in the [parameters](#) defined by the version of the `did:webvh` specification being used by the [DID Controller](#).
4. `base58btc` is an implementation of the [base58btc](#) function. Its output is the base58 encoded string of its input.

The following is an example of a preliminary [log entry](#) that is processed to produce an [entry hash](#). As this is a first entry in a [DID Log](#), the input `versionId` is the [SCID](#) of the DID.

```
{"versionId": "QmdmPkUdYzbr9txmx8gM2rsHPgr5L6m3gHjJGAf4vUFoGE", "versionTime": "2025-0
```

Resulting [entryHash](#): QmQ6FJ4fk2xheSSQoEjVpTgx9AQPkhJgtR9hn1nr4EeCrZ

§ Verify The Entry Hash

To verify the [entryHash](#) for a given [did:webvh](#) [DID log entry](#), a DID Resolver **MUST** execute the following process:

1. Extract the [versionId](#) in the [DID log entry](#), and remove from it the version number and dash prefix, leaving the log entry [entryHash](#) value.
2. Determine the hash algorithm used by the [DID Controller](#) from the [multihash](#) [entryHash](#) value.
 - The hash algorithm **MUST** be one listed in the [parameters](#) defined by the version of the [did:webvh](#) specification being used by the [DID Controller](#) based on the [method parameters](#) property set in the current or most recent prior [log entry](#).
3. Remove the [Data Integrity](#) [proof](#) from the [log entry](#).
4. Set the [versionId](#) in the entry object to be the [versionId](#) from the previous [log entry](#). If this is the first entry in the log, set the value to `<scid>`, the value of the [SCID](#) of the DID.
5. Calculate the hash string as `base58btc(multihash(JCS(entry), <hash algorithm>))`, where:
 1. [entry](#) is the data from the previous step.
 2. [JCS](#) is an implementation of the [JSON Canonicalization Scheme](#) ([RFC8785]). Its output is a canonicalized representation of its input.
 3. [multihash](#) is an implementation of the [multihash](#) specification. Its output is a hash of the input using the associated `<hash algorithm>`, prefixed with a hash algorithm identifier and the hash size.
 4. `<hash algorithm>` is the hash algorithm from Step 2.
 5. [base58btc](#) is an implementation of the [base58btc](#) function. Its output is the base58 encoded string of its input.
6. Verify that the calculated value matches the extracted [entryHash](#) value from Step 1. If not, terminate the resolution process with an error.

§ 3.7.5 Authorized Keys

Each entry in the [DID Log](#) **MUST** include a [Data Integrity](#) [proof](#) property signed by a key **authorized** to control (create, update, deactivate) the DID, and with the [proofPurpose](#) set to [assertionMethod](#). The authorized verification keys for [did:webvh](#) are the [multikey](#)-

formatted public keys in the **active** `updateKeys` list from the `parameters` property of the [log entries](#). Any of the authorized verification keys may be referenced in the [Data Integrity](#) proof.

For the first [log entry](#) the **active** `updateKeys` list is the one in that first [log entry](#).

A resolver of the DID **MUST** verify the signature and the key used for signing each [DID Log](#) entry **MUST** be one from the list of active `updateKeys`. If not, terminate the resolution process with an error.

The `did:webvh` Implementation Guide contains further discussion on the management of keys authorized to update the DID.

The **active** `updateKeys` for subsequent [entries](#) depends if the [Pre-Rotation](#) is active or not.

§ No Key Prerotation

For all subsequent [entries](#), the **active** list is the most recent `updateKeys` **before** the [log entry](#) to be verified. Thus, the general case is that each [log entry](#) is signed by the keys from the **previous** [log entry](#). Once a [log entry](#) containing an `updateKeys` list is published, that `updateKeys` becomes the active list, and previous `updateKeys` are ignored.

§ Pre-rotation

For all subsequent [entries](#), the **active** list is the `updateKeys` from the **current** [log entry](#) to be verified. Thus, the general case is that each [log entry](#) is signed by the keys from the **current** [log entry](#).

§ 3.7.6 DID Portability

As noted in the [Update \(rotate\)](#) section of the specification, a `did:webvh` DID can be renamed by changing the `id` DID string in the DIDDoc to one that resolves to a different HTTPS URL if the following conditions are met.

- The [DID Log](#) of the renamed DID **MUST** contain all of the [log entries](#) from the creation of the DID.
- The [log entry](#) in which the DID is renamed **MUST** be a valid DID entry building on the prior [DID log entries](#), per this specification.
- The [parameter](#) `portable` **MUST** be set to `true`, as defined in the [DID Method Parameters](#) section.
- The [SCID](#) **MUST** be the same in the original and renamed DID.
- The [DIDDoc](#) **MUST** contain the prior DID string as an `alsoKnownAs` entry.
- [DID Controllers](#) **SHOULD** account for any DNS requirements in making domain changes that impact a `did:webvh` DID being moved, such as those outlined in [\[\[spec:1034\]\]](#)

(“Domain Names - Concepts and Facilities”), and [\[RFC1035\]](#) (“Domain Names Implementation and Specification”).

Security Note – Misleading Prior Domain Association

When using portability, a `did:webvh` identifier may include a domain component that was never actually used to host its DID Log before being “moved” to a domain under the [DID Controller](#)’s control. This creates a potential for misleading claims of association with the original domain. Resolvers and clients of resolvers **MUST** ignore any prior domain components when evaluating the history or trustworthiness of a `did:webvh` DID; only the current hosting location and its associated verifiable history are relevant. In addition, the [whois](#) DID URL capability can be used to obtain attestations about the DID and [DID Controller](#) from relevant authorities.

§ 3.7.7 Pre-Rotation Key Hash Generation and Verification

Pre-rotation requires a [DID Controller](#) to commit to the authorization keys that will be used (“rotated to”) in the next [log entry](#) for updating the [DIDDoc](#). The purpose of committing to future keys is that if the currently authorized keys are compromised by an attacker, the attacker should not be able to take control of the DID by using the compromised keys to rotate to new keys the attacker controls. Assuming the attacker has not also compromised the committed key pairs, they cannot rotate the authorization keys without detection. See the non-normative section about [Pre-Rotation](#)[Using [Pre-Rotation](#) Keys]([#using-pre-rotation-keys](#)) in the `did:webvh` Implementer’s Guide for additional guidance.

As described in the [parameters](#) section of this specification, a [DID Controller](#) **MAY** include the [parameter](#) `nextKeyHashes` with a non-empty list in any [DID log entry](#) to activate the [pre-rotation](#) feature. When [pre-rotation](#) is active, all [multikey](#) representations of the public keys in the `updateKeys` [parameters](#) property in other than the initial version of the [DID log](#) **MUST** have their hash in the `nextKeyHashes` array from the previous [DID log entry](#). If not, terminate the resolution process with an error.

A [DID Controller](#) may turn off the use of pre-rotation by setting the [parameter](#) `nextKeyHashes` to `[]` (empty array) in any [DID log entry](#). If there is an active set of `nextKeyHashes` at the time, the pre-rotation requirements remains in effect for the [DID Log entry](#). The subsequent [DID Log entry](#) **MUST** use the non-pre-rotation rules.

To create a hash to be included in the `nextKeyHashes` array, the [DID](#) **MUST** execute the following process for each possible future authorization key.

1. Generate a new key pair. The key type **MUST** be one that can be used as a `did:webvh` authorization key.
2. Generate a [multikey](#) representation of the public key of the new key pair.
3. Calculate the hash string as `base58btc(multihash(multikey))`, where:

1. **multikey** is the [multikey](#) representation of the public key from Step 2.
2. **multihash** is an implementation of the [multihash](#) specification. Its output is a hash of the input using the associated **<hash algorithm>**, prefixed with a hash algorithm identifier and the hash size.
3. **<hash algorithm>** is the hash algorithm used by the [DID Controller](#). The hash algorithm **MUST** be one listed in the [parameters](#) defined by the version of the **did:webvh** specification being used by the [DID Controller](#).
4. **base58btc** is an implementation of the [base58btc](#) function. Its output is the base58 encoded string of its input.
4. Insert the calculated hash into the **nextKeyHashes** array being built up within the [parameters](#) property.
5. The generated key pair **SHOULD** be safely stored so that it can be used in the next [log entry](#) to become a DID authorization key. At that time, the [multikey](#) representation of the public key will be inserted into the **updateKeys** property in the [parameters](#) and the private key can be used to sign the [log entry](#)'s DID update authorizations proofs.

A [DID Controller](#) **MAY** add include extra entries (for keys or just random strings) in a **nextKeyHashes** array.

After rotating from a pre-rotation public key, the corresponding private key **SHOULD** be treated as **spent** and **securely destroyed**. Reusing a revealed pre-rotation key is strongly discouraged because it weakens the intended containment and forward-security properties of pre-rotation.

When processing other than the first [DID log entry](#) where [pre-rotation](#) feature is active, a **did:webvh** resolver **MUST**:

1. For each [multikey](#) in the **updateKeys** property in the **parameters** of the [log entry](#), calculate the hash and hash algorithm for the [multihash multikey](#).
2. The hash algorithm **MUST** be one listed in the [parameters](#) defined by the version of the **did:webvh** specification being used by the [DID Controller](#).
3. The resultant hash **MUST** be in the **nextKeyHashes** array from the previous [log entry](#) prior to being processed. If not, terminate the resolution process with an error.
4. A new **nextKeyHashes** list **MUST** be in the **parameters** of the [log entry](#) currently being processed. If not, terminate the resolution process with an error.

§ 3.7.8 DID Witnesses

The [witness](#) process for a **did:webvh** DID provides a way for collaborators to work with the [DID Controller](#) to “witness” the publication of new versions of the DID. This specification defines the technical mechanism for using [witnesses](#). Governance and policy questions about when and how to use the technical mechanism are outside the scope of this specification.

Witnesses can prevent a [DID Controller](#) from updating/removing versions of a DID without detection by the witnesses. [Witnesses](#) are also a further mitigation against malicious actors compromising both a [DID Controller](#)'s authorization key(s) to update the DID, and the [DID](#)'s web site where the [DID log](#) is published. With both compromises, a malicious actor might be able to take control over the DID by rewriting the [DID Log](#) using the keys they have compromised. By adding [witnesses](#) to monitor and approve each version update, a malicious actor cannot rewrite the previous history without having compromised a sufficient number of [witnesses](#), the [DID Controller](#)'s key(s), and the Web Server on which the [DID Log](#) is published.

§ Witness Lists

The list of DIDs that witness DID updates are defined in the `witness` parameter, as described in the [Parameters](#) section of this specification. After the first `witness` parameter has been set to other than `{}` (empty object) in a [DID log entry](#), and while there are active witnesses, a [threshold](#) of the active witnesses must provide valid proofs associated with each [DID log entry](#) before the [DID](#) can be published. If a [DID log entry](#) contains a new (replacement) list of witnesses (by including a new `witness parameter`) that new list becomes active **AFTER** the new [DID log](#) has been published. Such a replacement **MAY** be a `{}` (empty object). Once the `witness` attribute set to `{}` becomes active, updates to the DID are not [witnessed](#).

§ Witness DIDs and Reputation

Since `did:webvh` witness DIDs must be `did:key` DIDs, there is not an explicitly published identifier for each witness. If there is a need in an ecosystem to identify who the witnesses are, a mechanism should be defined by the governance of the ecosystem, such as the entry of the DID in a trust registry. Such mechanisms are outside the scope of this specification.

§ The `witness` Parameter

The `witness` element in a [parameters](#) object of a [DID Log entry](#) has the following data structure:

```
"witness" : {
  "threshold": n,
  "witnesses" : [
    {
      "id": "<did:key DID of witness>"
    }
  ]
}
```

where:

- **threshold**: an integer that must be attained or surpassed by the count of the witnesses for a DID log entry to be considered approved. **threshold** **MUST** be between 1 and the number of items in the **witnesses** array, inclusive.
- **witnesses** : the array of `[[ref; witnesses]]` that **MUST** be non-empty, with each entry including the field:
 - **id** : (required) the DID of the witness. The DID **MUST** be a did:key DID, and **MUST** be unique within the array (i.e., no duplicate witness DIDs).

§ Witness Threshold Algorithm

The use of the [threshold](#) versus needing approvals from all [witnesses](#) is to prevent faulty [witnesses](#) from blocking the publishing of a new version of the DID. To determine if the [threshold](#) has been met, all participants **MUST** count the verified [witness](#) approvals, and if it is equal to or more than the **threshold**, the update **MUST** be accepted as "[witnessed](#)".

§ The Witness Proofs File

Proofs from [witnesses](#) are placed into a separate file (`did-witness.json`) from the [DID Log](#). The same [DID to HTTPS Transformation](#) used for the [DID Log](#) is used to locate the `did-witness.json` resource, with only the last element changed (`did.jsonl` to `did-witness.json`). The media type of the file **SHOULD** be `application/json`.

The data model for the `did-witness.json` file is:

```
[
  {
    "versionId": "1-Qmba111111...",
    "proof": [{ ... }, { ... }]
  },
  {
    "versionId": "2-Qzmb222222...",
    "proof": [{ ... }, { ... }]
  }
]
```

Where:

- **versionId** is the **versionId** of the [DID log entry](#) to which the [witness](#) proofs apply.
- **proof** is an array of [Data Integrity](#) proofs that use the **versionId** as input data. The permitted [Data Integrity](#) cryptosuites used by the [witnesses](#) **MUST** be `eddsa-jcs-2022` as referenced in [spec:di-eddsa-v1.0] and the **proofPurpose** set to `assertionMethod`.

A valid proof from a [witness](#) carries the implication that **ALL** prior [DID Log entries](#) are also approved by that witness. To maintain a manageable `did-witness.json` file size, the [DID](#)

[Controller](#) **SHOULD** remove all older proofs for **published** [DID Log entries](#), keeping only the latest proof for each witness.

To eliminate the race condition in publishing the [DID Log](#) and `did-witness.json` files, when a new [DID Log entry](#) is being added, [witness](#) proofs **MUST** be added to the `did-witness.json` file and that file published **BEFORE** publishing the [DID Log](#) file containing the new [DID Log entry](#). As a result, `did:webvh` resolvers may find proofs for unpublished [DID log entries](#) in the `did-witness.json` file. Resolvers **MUST** ignore proofs with `versionId`s not in the [DID Log](#) file. Since resolvers cannot verify an unpublished [DID log entry](#), the [witness](#) proofs on unpublished [DID log entries](#) do not carry the implication of approval of prior [DID Log entries](#). Therefore, at times there may be two proofs in the `did-witness.json` file for a [witness](#):

- The most recent proof for a published [DID Log entry](#).
- An additional proof that applies to an unpublished [DID Log entry](#).

To avoid unnecessary clutter in the `did-witness.json` file, array entries without proofs (e.g., containing only the `versionId`) **SHOULD** be removed.

§ Witnessing a DID Version Update

The following process is used to witness a DID version update:

- The [DID Controller](#) prepares the full [DID Log Entry](#) (including the `proof` element) for the new version of the DID, and shares it with the active [witnesses](#).
 - The specification leaves to implementers *how* the [log entry](#) data is provided to the [witnesses](#).
- The [witnesses](#) ***MUST** hold their own copy of the published [DID Log](#) prior to witnessing a [DID Log entry](#).
- Each [witness](#) verifies the [DID Log Entry](#), as defined by this specification. If not verified, the [witnesses](#) **MUST NOT** approve the [log entry](#).
- Each [witness](#) determines (based on the governance of the ecosystem) if they approve of the DID version update.
 - The meaning of “approve” for any given implementation is outside the scope of this specification.
- If the verification is successful and approval granted, the [witness](#) creates and sends to the [DID Controller](#) a [Data Integrity](#) proof signed using the [witness](#)'s `did:key` DID.
 - The specification leaves to implementers how [witness](#) proofs are conveyed to the [DID Controller](#).
- The [DID Controller](#) **MUST** add the proof to the record for the applicable `versionId` for the unpublished [DID log entry](#). to the `did-witness.json` file.

- The [DID Controller](#) **MAY** publish the updated `did-witness.json` file as new witness proofs are added to the file.
- The [DID Controller](#) **MUST** publish the updated `did-witness.json` file **after** a [threshold](#) of witness proofs have been received and **before** the witnessed [DID Log](#) file is published.

§ Verifying Witness Proofs During Resolution

A `did:webvh` resolver **MUST** verify that all [DID Log entries](#) that have active [witnesses](#) have a [threshold](#) of active witnesses approving the [log entry](#). To do so, resolvers must:

- Successfully complete the non-[witness](#) verifications of the [DID Log](#).
- Retrieve the `did-witness.json` file.
- Verify enough of the [witness](#) proofs in the `did-witness.json` file to meet the [threshold](#) for all [DID log entries](#) requiring [witnessing](#).
 - The resolver **MUST** ignore the proofs of any unpublished [DID Log entries](#).
- For each [DID log entry](#) requiring witnessing, the resolver **MUST** confirm that the `did-witness.json` file contains verified [witness Data Integrity](#) proofs from a [threshold](#) of the then active [witnesses](#) for the current or any **later** published log entries. If not, terminate the resolution process with an error.

A [DID Controller](#) is expected to prune the `did-witness.json` file to include only the last proof for each witness for a published [DID log entry](#). However, if a [DID Controller](#) does not prune the file, a resolver **MAY** do the pruning as part of the resolution process, verifying only the minimum number of proofs needed to meet the [threshold](#) for each [DID log entry](#). While it is expected that a [DID Controller](#) will exclude any proofs that fail verification, a resolver **MAY** ignore any proofs that fail verification and still resolve the DID if there are enough valid proofs to meet the [threshold](#) requirements.

If you want to learn more about the practical application of witnesses, see the Implementer's Guide section on [Witnesses](#) on the `did:webvh` information site for more discussion on the witness capability and using it in production scenarios.

§ 3.7.9 DID Watchers

[Watchers](#) are components found in some digital trust and DID ecosystems that monitor DIDs on behalf of clients for various purposes, such as:

- **Caching verified DIDs:** Storing verified DID documents to facilitate efficient resolution.
- **Ensuring persistence:** Maintaining access to DIDs even after removal by the [DID Controller](#). For example, the  [Verifiable Data Gateway](#) in  `did:webplus` could function as a [watcher](#) for enduring DIDs.

- **Detecting inconsistencies:** Identifying malicious behavior by the [DID Controller](#), such as republishing altered [DID Logs](#). A network of [watchers](#) can reach consensus independently of [witnesses](#).

Any party may set up a [watcher](#) for `did:webvh` DIDs. However, a `did:webvh` DID Controller may opt to collaborate with specific [watchers](#) by publishing their URIs in the [parameters](#) of [DID log entries](#). It is outside the scope of this specification how a [DID controller](#) requests a [watcher](#) monitor a DID or how a [watcher](#) requests it be included in the [DID Log](#) of a DID.

The governance of [watchers](#) is out of scope for this specification, which defines only the technical mechanisms for notifying and querying [watcher](#) services.

§ Publishing Watcher URLs

`did:webvh` provides a mechanisms for notifying resolvers (and their clients via `[[spec:DID-RESOLUTION]]` metadata) about configured [watchers](#). The `watchers` [parameter](#) lists URIs that identify the DID's [watchers](#).

[Watchers](#) can be used by `did:webvh` resolvers and resolver clients. When resolving a `did:webvh` DID, `did:webvh` resolvers **MUST** provide the active list of [watchers](#) in the DID metadata, as noted in the [read/resolve](#) section of this specification.

Watcher URIs MAY use schemes other than HTTP(S), such as a DID (`[DID-CORE]`), depending on the specific implementation or network requirements. However, this specification does not define how non-HTTP(S) [watcher](#) URIs should be resolved or interacted with. If a [watcher](#) uses an HTTP(S) URL, it **MUST** support the HTTP-based interaction model defined in the [Watcher Endpoints and Behavior](#) section.

If a `watchers` entry is included in a [DID log entry](#), it replaces the active set of [watchers](#).

If a new [watcher](#) is added after a DID has existed for some time, the [DID Controller](#) **SHOULD** notify the new [watcher](#) about previously created [DID Resources](#).

[Watchers](#) do not need to be listed in the [DID log](#). [Watchers](#) can operate independently of the [DID Controller](#) by polling for updates. [DID Controllers](#) **MAY** send notifications to [watchers](#) that the [DID Controller](#) is aware of but does not list in the [DID Log](#).

§ Watcher Endpoints and Behavior

A [watcher](#) is a web server accessible via HTTP that **MUST** support the following capabilities:

- **Client Requests:**
 - Retrieve the [DID log](#) file for a given [SCID](#).
 - Retrieve the witness file for a given [SCID](#).
 - Retrieve a resource for a given [SCID](#) and resource path.

- **Notifications (typically) from the [DID Controller](#):**

- Notify the [watcher](#) about a new log entry for a DID.
- Notify the [watcher](#) about a new or updated [DID resource](#).
- Request removal of a given [SCID](#) from the [watcher](#)'s cache.
- Request removal of a given DID resource [SCID](#) from the [watcher](#)'s cache.

§ [Watcher HTTP API Operations](#)

The following HTTP API operations define the interaction between [watchers](#) and other components. Included with the specification is the [did:webvh v1.0 Watcher OpenAPI YML Definition](#) that includes the request and response data models and status codes for each of the endpoints.

- **GET [<WATCHER URL>/log?scid=<SCID>](#)** : Returns the latest [DID Log](#) for the given [SCID](#).
- **POST [<WATCHER URL>/log?did=<DID>](#)** : Notifies the [watcher](#) of a log update, prompting retrieval of the latest [DID Log](#) and [witness](#) file. This endpoint uses the [did](#) as the query parameter instead of the [SCID](#) to ensure that the [watcher](#) is notified in the case of the DID moving to a new web location. The [watcher](#) is expected to continue indexing the DID using its [SCID](#).
- **POST [<WATCHER URL>/log/delete?scid=<SCID>](#)** : Notifies the [watcher](#) that the given [<SCID>](#) should be deleted from the [watcher](#)'s cache. If removed, subsequent requests for that [<SCID>](#) from clients should return a **404 Not Found** status. The body of the URL is a Data Integrity proof from the requester that may be used by the [Watcher](#) to decide on the legitimacy of the request. The [Watcher](#) will act (or not) on the request according to its governance, which is out of scope of this specification. For example, a [watcher](#) might implement a workflow be completed to approve the deletion of a [<SCID>](#) from the [watcher](#)'s cache. The endpoint could be used to carry out a “right to be forgotten” order, such as might be required under Europe’s [General Data Protection Regulation \(GDPR\)](#).
- **GET [<WATCHER URL>/witness?scid=<SCID>](#)** : Returns the latest [witness.json](#) file for the given [SCID](#).
- **GET [<WATCHER URL>/resource?scid=<SCID>&path=<resourcePath>](#)** : Retrieves the requested resource.
- **POST [<WATCHER URL>/resource?scid=<SCID>&path=<resourcePath>](#)** : Notifies the [watcher](#) of a new or updated resource.
- **POST [<WATCHER URL>/resource/delete?scid=<SCID>&path=<resourcePath>](#)** : Notifies the [watcher](#) that the given [<resourcePath>](#) associated with the [<SCID>](#) should be deleted from the [watcher](#)'s cache. If removed, subsequent requests for that [<SCID>](#) and [<resourcePath>](#) from clients should return a **404 Not Found** status. The body of the URL is a Data Integrity proof from the requester that may be used by the [Watcher](#) to decide on the legitimacy of the request. The [Watcher](#) will act (or not) on the request

according to its governance, which is out of scope of this specification. For example, a [watcher](#) might implement a workflow be completed to approve the deletion of a `<SCID>` from the [watcher](#)'s cache. The endpoint could be used to carry out a “right to be forgotten” order, such as might be required under Europe’s [General Data Protection Regulation \(GDPR\)](#).

§ 3.7.10 Publishing a Parallel `did:web` DID

Each time a `did:webvh` version is created, the [DID Controller](#) **MAY** generate a corresponding `did:web` to publish along with the `did:webvh`. If this is being done, the `did:webvh` DIDDoc **SHOULD** have the corresponding `did:web` in the `alsoKnownAs` array. To publish a parallel `did:web` DIDDoc, the [DID Controller](#) **MUST**:

1. Start with the resolved version of the [DIDDoc](#) from `did:webvh`.
2. If the “implicit” `did:webvh` services (as defined in the [DID URL Resolution](#) section) are not already present in the [DIDDoc](#), they **MUST** be added. These services are the `relativeRef` service with `id: "#files"` or `id: "<did>#files"` and the `whois` service with `id: "#whois"` or `id: "<did>#whois"`, with the `serviceEndpoint` for both derived from the [DID-to-HTTPS transformation](#).
3. Execute a text replacement across the [DIDDoc](#) of `did:webvh:<SCID>:` to `did:web:`, where `<scid>` is the actual `did:webvh` [SCID](#).
4. Add to the [DIDDoc](#) `alsoKnownAs` array, the full `did:webvh` DID. If the `alsoKnownAs` array does not exist in the [DIDDoc](#), it **MUST** be added.
5. Remove any duplicate entries in the `alsoKnownAs` array, including the `did:web` DID itself if it was duplicated in the earlier steps.
6. Publish the resulting [DIDDoc](#) as the file `did.json` at the web location determined by the specified `did:web` DID-to-HTTPS transformation.

The benefit of doing this is that resolvers that have not been updated to support `did:webvh` can continue to resolve the [DID Controller](#)'s DIDs. `did:web` resolvers that are aware of `did:webvh` features can use that knowledge, and the existence of the `alsoKnownAs` `did:webvh` data in the [DIDDoc](#) to get the verifiable history of the DID.

The risk of publishing the `did:web` in parallel with the `did:webvh` is that the added security and convenience of using `did:webvh` are lost.

§ 3.8 DID URL Resolution

The `did:webvh` DID method embraces the expressive power of DID URLs while preserving the semantic simplicity of a web-based resolution model. In particular, `did:webvh` implementations **MUST** support path-based DID URL resolution in a manner consistent with the [DID Core specification](#).

Specifically, a `did:webvh` resolver **MUST**:

- Resolve any `did:webvh` DID URL with a path component using an implicit `relativeRef` service as defined in [DID-CORE]. The path is appended directly to the HTTPS URL obtained from the [DID-to-HTTPS transformation](#), excluding any `.well-known` prefix.
 - For example, resolving `did:webvh:{SCID}:example.com/governance/issuers.json` retrieves the file located at `https://example.com/governance/issuers.json`.
 - This behavior can be overridden by defining an explicit service in the DID Document.
- Resolve the special path `/whois` using an implicit `[[spec:LINKED-VP]]` service. This applies regardless of whether a `whois` service is explicitly defined in the `DIDDoc`. The resolver **MUST** retrieve the Verifiable Presentation, if published by the [DID Controller](#), from the web location corresponding to the DID-to-HTTPS transformation (excluding `.well-known`), using the path `/whois.vp` and media type `application/vp` as registered in the [IANA Media Types Registry](#).
 - For example, resolving `did:webvh:{SCID}:example.com/whois` returns the content of `https://example.com/whois.vp` if available.

In both cases, a [DID Controller](#) **MAY** override the implicit resolution behavior by defining explicit services in the DID Document, which take precedence over the defaults.

The sections below formalize the structure and resolution rules for each default service and describe how they may be overridden by the [DID Controller](#).

§ 3.9 DID URL Path Resolution

The automatic resolution of `did:webvh` DID URL paths follows the [DID-CORE] `relativeRef` mechanism, enabling path-based access to web resources directly tied to the DID's domain. The approach is derived from Examples 2 and 8 in [Section 3.2 of DID Core](#):

- A DID URL such as `did:example:123456/resume.pdf` (see Example 2) is semantically equivalent to:
`did:example:123456?service=files&relativeRef=/resume.pdf` (see Example 8).
- The `service=files` reference resolves against a DID Document service with `"id": "#files"` or `"id": "<did>#files"` and a `type` of `relativeRef`.

The `did:webvh` method implicitly defines this service, with a `serviceEndpoint` derived from the [DID-to-HTTPS transformation](#). The final path segment (`did.jsonl`) is replaced by the DID URL path. If the resulting HTTPS URL contains `.well-known/`, that segment **MUST** be removed before dereferencing the resource.

For example, the following is the implicit service definition for the DID `did:<scid>:webvh:example.com`:

```
{
  "id": "#files",
  "type": "relativeRef",
  "serviceEndpoint": "https://example.com/"
}
```

A [DID Controller](#) **MAY** explicitly define a service entry with `"id": "#files"` in the [DIDDoc](#). If present, this **MUST** override the implicit service described above. `id` can be an absolute reference that includes the DID with the `#files` fragment (`<did>#files`), or a relative reference as above.

To resolve a DID URL of the form `<did:webvh DID>/path/to/file`, a `did:webvh` resolver **MUST**:

1. Resolve the base `did:webvh` DID by retrieving, verifying, and processing its [DID Log](#), as defined in this specification.
2. Locate the service entry with `id "#files"` or `"<did>#files"` in the resulting [DIDDoc](#), or fall back to the implicit service if none is defined.
3. Construct the URL by appending the DID URL path to the `serviceEndpoint`, and attempt to retrieve the resource from that location.
 - If the scheme of the `serviceEndpoint` is not supported by the resolver (e.g., non-HTTP(S) protocol), the resolver **MUST** return an `invalidDid` error.
 - If the resolution of the constructed URL fails with a “not found” condition (e.g., HTTP 404), the resolver **MUST** return the `notFound` error.

§ 3.9.1 whois LinkedVP Service

§ 3.10 WHOIS Resolution

The `#whois` service enables recipients of a `did:webvh` DID to retrieve a [Verifiable Presentation](#)—optionally published by the [DID](#)—containing one or more embedded [Verifiable Credentials](#). These credentials may help resolvers or relying parties make informed trust decisions about the controller of the DID.

The intention is that resolving `<did:webvh DID>/whois` yields a [Verifiable Presentation](#) published by the [DID Controller](#) that includes credentials with the DID as the `credentialSubject`. The contents of the presentation are determined solely by the [DID Controller](#), who selects which credentials to include. It is up to the resolver or relying party to decide what assertions (and issuers) are relevant for establishing trust.

`did:webvh` DIDs **automatically** support a `/whois` service endpoint, implicitly defined using the `[[spec:LINKED-VP]]` service type. The `serviceEndpoint` is computed using the standard

[DID-to-HTTPS transformation](#), replacing `did.jsonl` with `whois.vp`, and omitting any `.well-known/` prefix.

The default `#whois` service is:

```
{
  "@context": "https://identity.foundation/linked-vp/contexts/v1",
  "id": "#whois",
  "type": "LinkedVerifiablePresentation",
  "serviceEndpoint": "<did-to-https-translation>/whois.vp"
}
```

The file located at the `serviceEndpoint` **MUST** contain a [Verifiable](#) conforming to the [W3C VCDM](#). It **MUST** be signed by the DID. `id` can be an absolute reference that includes the DID with the `#whois` fragment (`<did>#whois`), or a relative reference as above.

The presentation **MUST** include at least one [Verifiable Credential](#) where the `credentialSubject.id` is the DID. Such [Verifiable](#) might serve to bind the DID to other identifiers associated with the [DID Controller](#). Additional credentials in the presentation **MAY** be associated with those other identifiers, rather than the DID itself. For example, the presentation could include a credential linking the DID for a business to that business's registration ID, and a second credential (perhaps an ISO certification) where the `credentialSubject.id` is the registration ID.

A [DID Controller](#) **MAY** explicitly define a `service` with `"id": "#whois"` in the [DIDDoc](#). If present, this entry **MUST** override the implicit service defined above. This is required if the controller wishes to:

- Publish the WHOIS Verifiable Presentation in a different format (i.e., not [W3C VCDM](#))
- Serve the WHOIS presentation from a different location or using a non-default media type

To resolve the DID URL `<did:webvh DID>/whois`, a resolver **MUST**:

1. Resolve the base `did:webvh` DID by retrieving, verifying, and processing the [DID Log](#). The resolver will use either an explicitly defined service with `"id": "#whois"` or `"id": "<did>#whois"`, or the implicit service defined above.
2. Construct and attempt to retrieve the resource from the `serviceEndpoint` URL.
 - If the scheme of the `serviceEndpoint` is unsupported by the resolver (e.g., non-HTTP(S)), the resolver **MUST** return the `invalidDid` error.
 - If the request to the service endpoint results in a "not found" condition (e.g., HTTP 404), the resolver **MUST** return the `notFound` error.

The returned `whois.vp` **MUST** contain a [W3C VCDM verifiable](#) signed by the DID and containing [verifiable credentials](#) that **MUST** have the DID as the `credentialSubject`.

If a [DID Controller](#) publishes a parallel `did:web` DID and a `whois.vp` file, the `/whois` endpoint can be resolved using either DID, returning the same content either way. The [verifiable presentation](#) proof can reference either DID or include two proofs, each referencing a verification method for one of the DIDs. If only one DID is referenced, since both DIDs will have an `alsoKnownAs` for one another and include the same verification methods, a resolver using the DID not referenced in the proof can choose to verify the proof with the already resolved DID, or resolve the referenced DID before verifying the proof.

A [DID Controller](#) **MAY** explicitly add to their [DIDDoc](#) a `did:webvh` service with the `"id": "#whois"` or `"id": "<did>#whois"`. Such an entry **MUST** override the implicit `service` above. If the [DID](#) wants to publish the `whois` [verifiable presentation](#) in a different format than the [W3C VCDM](#) format, they **MUST** explicitly add to their [DIDDoc](#) a service with the `"id": "#whois"` or `"id": "<did>#whois"` to specify the name and implied format of the [verifiable](#).

§ 4. Security Considerations

This section follows the guidelines in [\[RFC3552\]](#) and addresses the security requirements for all DID operations defined in the `did:webvh` method specification. It draws on the method-specific characteristics of `did:webvh` while aligning with the [\[DID-CORE\]](#) requirements in [DID Core 7.3](#).

§ 4.1 Threats and Attacks

Implementations of `did:webvh` **MUST** mitigate the following classes of attack for all DID operations:

- **Eavesdropping** — All network communication (e.g., retrieval of `did.jsonl`, `witness.json` files, or other DID-associated resources) **SHOULD** be performed over TLS (HTTPS). Plaintext HTTP **MUST NOT** be used except for testing or non-production deployments where confidentiality is not required. This requirement is not unique to the `did:webvh` method; its application here is as with general web traffic. The verifiability of `did:webvh` ensures that tampering with the contents of individual log entries is detectable, TLS protects against passive observation and other network-based risks.
- **Replay attacks** — Implementations **MUST** verify the DID Log (including monotonic progression and referenced hashes) and reject logs that do not verify as outlined in the [Read](#) section of this specification.
- **Message insertion, deletion, and modification** — Each DID Log entry and witness file is integrity-protected using cryptographic signatures. Implementations **MUST** verify the DID

Log and reject any log that fails verification as outlined in the [Read](#) section of this specification.

- **Truncation or withholding of log entries** — An attacker (or misconfigured intermediary) could serve an older-but-valid prefix of the DID Log, truncating newer entries and presenting a stale DID state.
 - **Mitigations (non-exhaustive):**
 - **Resolver cache-and-compare:** Resolvers **SHOULD** remember the latest `versionId` previously observed for a DID and **SHOULD** warn or fail resolution when presented with a truncated log.
 - **Witness verification:** Where DID Controllers utilize witnesses, resolvers **MUST** verify that all entries are properly witnessed according to the [Witness Verification](#) section of this specification. Notably, witness proofs of unpublished or truncated entries **MUST** be ignored.
 - **Multi-source resolution:** Resolvers **MAY** attempt retrieval from multiple [did:webvh Watchers](#) and detect divergence in latest entries.
 - **End-to-end TLS:** While signatures detect per-entry tampering, use of TLS **SHOULD** be enforced to reduce opportunities for active truncation in transit.
- **Denial of Service (DoS) and amplification** — Implementations **MAY** limit resource consumption by rejecting excessively large DID logs and witness proof files, and throttling repeated resolution attempts. The subjective elements of these limits (e.g., what constitutes “excessively large” or “repeated”) are ecosystem-dependent and **SHOULD** be established through community or ecosystem governance.
- **Man-in-the-middle (MitM)** — HTTPS and signature verification of DID Log entries and witness proofs protect against undetected modification of individual entries. Risks specific to withholding or truncation are addressed in **Truncation or withholding of log entries** above. Use of TLS further ensures server authenticity and reduces opportunities for active interference.
- **Conflicting parallel updates / split view** — Multiple authorized updates from the same parent entry can produce divergent logs. Publication components **MUST** enforce monotonic extension of the current tip; witnesses **MUST NOT** sign more than one child of the same parent; consolidation of `witness.json` **MUST** fail on conflicting branches. Resolvers **SHOULD** cache the highest observed version/hash and **SHOULD** detect/warn on older branches; Watchers **SHOULD** detect and report divergence across sources.
- **Other attacks** — The required [did:webvh](#) verification process mitigates downgrade attacks on cryptographic algorithms and prevents poisoning of log or witness files, since unauthorized changes fail signature verification. Verification does not, however, address availability risks; implementers **SHOULD** consider operational measures (e.g., [watchers](#) and well-known web techniques) to improve resilience.

- **Misleading prior-domain association** — A DID may be ported from a domain it never actually used, creating a false impression of association with that domain. Mitigation: resolvers and clients **MUST** ignore prior domain components when evaluating the DID, as described in [Unique Assignment of DIDs](#).

The use of DNSSEC [\[RFC4033\]](#), [\[RFC4034\]](#), [\[RFC4035\]](#) is essential to prevent spoofing and ensure authenticity of DNS records.

§ 4.2 Residual Risks

Residual risks include:

- Compromise of the web hosting infrastructure serving the DID resources.
 - While this can impact access to the DID Log and associated files, it does not compromise the integrity of the log entries themselves, nor the verifiability of the DID.
- Compromise of controller private keys.
 - A `did:webvh` [DID Controller](#) can mitigate this risk through the use of [pre-rotation keys](#).
 - In doing so, [DID Controller](#)'s **SHOULD** avoid reusing revealed pre-rotation keys. While not invalid per this specification, re-use of a pre-rotation key after disclosure reduces compromise containment. Mitigation: follow the one-time-use best practice and securely destroy revealed private keys (see [Pre-rotation Key Hash Generation and Verification](#)). Resolvers are **NOT REQUIRED** to enforce this, but **MAY** warn.
 - Additional good security practices **SHOULD** also be followed, such as using hardware security modules (HSMs) or secure enclaves for key storage, enforcing strong access controls, maintaining secure backups of critical keys, and performing regular key rotations.
- Weaknesses in underlying cryptographic algorithms after deployment.
- Misconfiguration of cache control or TTL values.
- Implementation errors in DID resolvers or controllers.

§ 4.3 Integrity Protection and Update Authentication

All DID operations (create, update, deactivate) are integrity-protected by the cryptographic verification of DID Log entries. Update authentication is provided by verifying the [DID Controller](#)'s proof(s) against the valid `updateKeys` in the DID [Parameters](#).

Because a `did:webvh` DID document and its associated log entries are self-certifying, they can be verified and trusted regardless of how they are retrieved — whether directly from the host, via a cache, through a CDN, from a [DID Watcher](#), or via a trusted resolver service. The verification process ensures authenticity and integrity independent of the transport channel.

§ 4.4 Authentication Characteristics

The authentication of DID updates is based on possession of the private keys associated with the update and pre-rotation keys. The security of the DID therefore depends on the strength of these keys, their secure storage, and the cryptographic algorithms used.

§ 4.5 Unique Assignment of DIDs

In `did:webvh`, uniqueness of a DID is based on the [self-certifying identifier](#) (SCID) generated at the inception of the DID. The SCID is cryptographically bound to the DID Controller's keys and ensures that no two independently created DIDs can have the same identifier.

The DNS portion of the DID is used solely for discovery of the DID Log and associated files; it is not used for verification of DID control. Further, the DNS name does not need to be owned or directly controlled by the DID Controller. For example, a DID can be published within a namespace provided by a hosting platform (e.g., a GitHub repository or pages site) that serves static files over HTTPS. In such cases, platform policies and HTTPS server authentication are relied upon for access and integrity at the transport layer, while DID verification is provided entirely by the SCID and verifiable history of the DID.

A `did:webvh` identifier may include a domain component that was never actually used to host its DID Log, before being moved — via the [did:webvh portability](#) capability — to a different domain under the [DID Controller](#)'s control. This creates a potential for misleading claims of association with the original domain. To prevent this, resolvers and clients of resolvers **MUST** ignore any prior domain components when evaluating the history or trustworthiness of a `did:webvh` DID; only the current hosting location and its associated verifiable history are relevant. In addition, the [whois](#) DID URL capability **SHOULD** be used to obtain attestations about the DID and [DID Controller](#) from relevant authorities.

§ 4.6 Endpoint Authentication

DID resource retrieval endpoints **MUST** be authenticated using TLS server authentication. Self-signed certificates **SHOULD NOT** be used in production. This requirement is not unique to the `did:webvh` method; it applies to all web traffic. While the verifiability of `did:webvh` ensures that any tampering with the contents of individual log entries is detectable, TLS provides additional protection against active network attacks (including truncation or withholding) and ensures the authenticity of the server providing the DID resources.

§ 4.7 Network Topology

Unlike DLT-based DID methods, `did:webvh` relies on web infrastructure and does not require peer-to-peer networking. However, implementations relying on CDN caching or load balancers **MUST** ensure these intermediaries do not serve stale or tampered DID data.

§ 4.8 Cryptographic Protection

The following data is protected:

- **DID Log entries** — Signed by the DID controller's keys.
- **Witness proofs** — Signed by witness nodes' keys.

These signatures provide integrity and update authentication but not confidentiality; DID Log entries are public.

Secret data (e.g., controller private keys, witness private keys, random seeds) **MUST** be protected in secure storage and never exposed in the DID Log.

§ 4.9 Signature Implementation

did:webvh uses standard [Data Integrity](#) proof mechanisms for signing DID Log entries and witness proofs, as defined in the cryptographic suite used. Implementations **MUST** follow the suite's signature generation and verification requirements.

§ 4.10 International Domain Names

did:webvh implementers **MAY** publish [DID Logs](#) on domains that use international domains. The [DID-to-HTTPS Transformation](#) section of this specification **MUST** be followed by [DID Controllers](#) and DID resolvers to ensure the proper handling of international domains.

§ 4.11 Cross-Origin Resource Sharing (CORS) Policy Considerations

To support scenarios where DID resolution is performed by client applications running in a web browser, the file served for the [DID Log](#) needs to be accessible by any origin. To enable this, the [DID Log](#) HTTP response **MUST** include the following header:

```
Access-Control-Allow-Origin: *
```

§ 4.12 Publishing parallel **did:web**

did:webvh implementers that consider [publishing parallel did:web DID](#) **SHOULD** evaluate security impact from losing added security properties of **did:webvh** and refer to [did:web Security and Privacy Considerations](#) for additional guidance.

§ 4.13 Post Quantum Attacks

did:webvh [Key Pre-Rotation](#) approach provides enough flexibility for “post-quantum safety”. For guidance on post-quantum attacks mitigation, implementors **SHOULD** refer to [corresponding Implementation Guide section](#).

§ 5. Privacy Considerations

This section addresses the privacy considerations in alignment with [\[RFC6973\]](#) Section 5 and the [\[DID-CORE\]](#) requirements in [DID Core 7.4](#).

§ 5.1 Surveillance

The `did:webvh` method publishes DID logs to publicly accessible HTTPS endpoints. While the contents of the DID Log are generally intended to be public, the timing, frequency, and correlation of updates can be observed and may reveal operational patterns or associations.

Resolution of a `did:webvh` identifier also exposes the resolver's network activity to DNS providers and web servers, which could be used for tracking. Controllers and resolvers **MAY** use privacy-enhancing technologies such as VPNs, TOR, or trusted universal resolver services, and **MAY** adopt emerging approaches such as [Oblivious DNS over HTTPS \(ODoH\)](#) to reduce this risk.

§ 5.2 Stored Data Compromise

DID data is stored on web servers. A compromise of the hosting infrastructure could allow tampering with DID resources. HTTPS and cryptographic signatures protect integrity, but confidentiality is not provided.

§ 5.3 Unsolicited Traffic

Publishing a DID Log does not inherently solicit inbound traffic beyond normal DID resolution. However, public exposure of service endpoints in the DID Document may increase unsolicited interactions. [DID Controllers](#) **SHOULD** avoid publishing unnecessary endpoints.

§ 5.4 Misattribution

Because the DNS portion of the DID is used for discovery, a misattribution risk arises if that DNS name is reassigned without the associated DID resources being updated or removed. Controllers **SHOULD** ensure DID deactivation before relinquishing a DNS name or namespace.

Where possible, Controllers **SHOULD** use the [DID Portability](#) mechanism defined in this specification to move the DID to a new location under their control. When portability is used, an HTTP redirect from the old location to the new one is the preferred approach, even in cases where DID ownership is transferred, as it enables seamless resolution while preserving the DID's verifiable history.

§ 5.5 Correlation

The use of a static DID and public DID Log entries can enable correlation of activities over time. Controllers **SHOULD** avoid embedding personal identifiers or unnecessary service endpoints in DID documents.

§ 5.6 Identification

DIDs are public identifiers and can be linked to real-world identities through their domain ownership. Entities that require anonymity **SHOULD** consider DID methods designed for

pseudonymity.

§ 5.7 Right to Erasure ([GDPR Art. 17](#))

While it's possible for [DID Controller](#) to delete published data as described in [Deactivate \(Revoke\) operation](#), it's **RECOMMENDED** for monitoring [watchers](#) to cache last known state indefinitely. This means that ability and specific process of complete data erasure depends on [watchers](#) behavior and **SHOULD** be defined by governance of ecosystem.

§ 5.8 Secondary Use

Information published in the DID Log may be repurposed by third parties. Controllers **SHOULD** minimize the publication of data that could be used for purposes beyond the intended use.

§ 5.9 Disclosure

All data in the DID Log is publicly accessible. Sensitive data **MUST NOT** be included.

§ 5.10 Exclusion

`did:webvh` uses DNS for discovery, and while the DID Controller may control the web server on which a `did:webvh` DID is published, the DID Method does **not** require controller ownership of a DNS domain. Controllers **MAY** publish the DID Log and associated resources under a namespace they control on a web-hosting platform that serves static files over HTTPS (for example, a GitHub repository or pages space). This reduces barriers to participation.

Residual exclusion risks remain: access to such platforms typically requires an account and compliance with provider terms of service; platforms might impose geoblocking, payment requirements, or content restrictions; and accounts can be suspended. Controllers **SHOULD** maintain the ability to republish or mirror DID resources under alternative hosts (including using `did:webvh` [Watchers](#)) and **SHOULD** document a transition plan so that participants are not locked out if a hosting provider becomes unavailable. The verifiable history of the DID ensures that the DID can be verified regardless of the source of the [DID Log](#) and related files.

§ 5.11 No Phone Home Mitigations

A privacy concern in decentralized identity ecosystems is the possibility of an issuer of identity information (such as Verifiable Credentials) being notified when and where individuals present those credentials. This “phone home” surveillance problem (such as described by [nophonehome.com](#)) can occur if the presentation of a credential requires contacting the issuer’s infrastructure, either directly or indirectly, in a way that can be linked to the credential holder. As `did:webvh` issuer DIDs may be self-hosted, this is particularly relevant.

While this concern is generally associated with the use of verifiable credentials rather than about the resolution of DIDs, a `did:webvh` server operated by an issuer might host related

resources that are retrieved at credential presentation time — for example, revocation registries or status lists. If these resources are implemented in a way that enables linking access patterns to individual credential holders, the [DID Controller](#) could use that information for surveillance.

Privacy-respecting credential issuers, credential holders, and verifiers all have a role in preventing “phone home” surveillance. The following practices can help:

- **Privacy-respecting Issuers (including DID Controllers hosting VC-related resources)**
 - **SHOULD NOT** design or deploy credential-related resources (such as revocation registries) in a way that enables the identification of individual holders at presentation time.
 - Use privacy-preserving designs — such as compact status lists, batching, and/or large revocation registries that provide “lost in a crowd” anonymity — to prevent correlation of access patterns to specific credential holders.
 - Use HTTP caching headers (e.g., **Cache-Control**, **ETag**) to enable CDNs, browsers, and resolvers to cache DID resources efficiently, reducing repeated origin requests that could enable tracking and improving performance.
- **Holders**
 - Use privacy-preserving techniques such as using [DID Watchers](#), trusted intermediaries, or privacy-enhancing network tools (e.g., TOR, VPN) to retrieve revocation status data without revealing the holder’s location or identity to the issuer.
 - Separate in time interactions with the issuer (e.g., retrieving revocation status) from credential presentations to verifiers, and structure retrievals to avoid creating identifiable access patterns that could enable correlation or surveillance.
- **Verifiers**
 - Be flexible in the timeliness of credential status checks, and consider omitting them entirely when risk is low, reducing or eliminating the need for the retrieval of status data.
 - Separate the retrieval of status or issuer data from the verification process by caching issuer-provided information where possible, so holders are not required to contact the issuer in real time.
 - Use privacy-enhancing network tools (e.g., TOR, VPN) or trusted intermediary resolvers to retrieve DID resources in a way that avoids revealing verifier identity or network location to the issuer or hosting provider.
 - Where possible, support privacy-preserving resolution protocols or intermediaries offered by the hosting party.

A related risk is that an issuer may deliberately or inadvertently create **holder-specific identifiers** for data elements that are expected to be common across all holders — for example, by issuing personalized revocation list URLs or unique resource paths. This enables tracking of specific holders even if the underlying credential is otherwise privacy-preserving. Preventing this requires shared responsibility: issuers **MUST NOT** generate such holder-specific identifiers; holders and verifiers **SHOULD** reject credentials or status mechanisms that contain them; and independent third parties, including [DID Watchers](#), **SHOULD** monitor issuer implementations to detect and report violations of this principle.

§ 6. Definitions

base58btc

Applies `[[spec:draft-msporny-base58-03]]` to convert data to a **base58** encoding. Used in **did:webvh** for encoding hashes for [SCIDs](#) and [entry hashes](#).

Data Integrity

[W3C Data Integrity](#) is a specification of mechanisms for ensuring the authenticity and integrity of structured digital documents using cryptography, such as digital signatures and other digital mathematical proofs.

Decentralized Identifier

Decentralized Identifiers (DIDs) [\[DID-CORE\]](#) are a type of identifier that enable verifiable, decentralized digital identities. A DID refers to any subject (e.g., a person, organization, thing, data model, abstract entity, etc.) as determined by the controller of the DID.

DID Controller

The entity that controls (create, updates, deletes) a given DID, as defined in the [\[DID-CORE\]](#).

DIDDoc

A DID Document as defined by the [\[DID-CORE\]](#) – the document returned when a DID is resolved.

DID Log

A DID Log is a list of [Entries](#), with an entry added for each update of the DID, including new versions of the [DIDDoc](#) or changed information necessary to generate or validate the DID.

DID Log Entry

A DID Log Entry is a JSON object that defines the authorized transformation of a [DIDDoc](#) from one version to the next. The initial entry establishes the DID and version 1 of the [DIDDoc](#). All entries are stored in the [DID Log](#).

DID Method

DID methods are the mechanism by which a particular type of DID and its associated DID document are created, resolved, updated, and deactivated. DID methods are defined using separate DID method specifications. This document is the DID Method Specification for `did:webvh`.

DID Portability

`did:webvh` portability is the capability to change the DID string for the DID while retaining the [SCID](#) and the history of the DID. This is useful when forced to change (such as when an organization is acquired by another, resulting in a change of domain names) and when changing DID hosting service providers.

DID Resources

A DID Resource is an object (often a file) that is referenced by a DID URL, with a path to the resource. The DID URL allows resolvers to locate and retrieve specific content associated with a DID. Examples include configuration files, schemas, credential definitions, or other structured data linked to the DID.

did:web

`did:web` as described in the [W3C specification](#) is a DID method that leverages the Domain Name System (DNS) to perform the DID operations. It is valued for its simplicity and ease of deployment compared to [DID methods](#) that are based on distributed ledgers or blockchain technology, but also comes with increased challenges related to trust, security and verifiability. `did:web` provides a starting point for `did:webvh`, which complements `did:web` with specific features to address its challenges while still providing ease of deployment.

Entry Hash

A `did:webvh` entry hash is a hash generated using a formally defined process over the input data to a [log entry](#), excluding the [Data Integrity](#) proof. The input data includes content from the predecessor to the version of the DID, ensuring that all the versions are “chained” together in a sort of microledger. The generated [entry hash](#) is subsequently included in the `versionId` of the [log entry](#) and **MUST** be verified by a resolver.

ISO8601

A date/time expressed using the [ISO8601 Standard](#).

JSON Canonicalization Scheme

[\[RFC8785\]](#) defines a method for canonicalizing a JSON structure such that is suitable for verifiable hashing or signing.

JSON Lines

A file of JSON Lines, as described on the site <https://jsonlines.org/>. In short, `JSONL` is lines of JSON with whitespace removed and separated by a newline that is convenient for handling streaming JSON data or log files.

Pre-Rotation

A technique for a controller of a cryptographic key to commit to the public key it will rotate to next, without exposing that actual public key. It protects from an attacker that gains knowledge of the current private key from being able to rotate to a new key known only to the attacker.

Linked-VP

A [\[DID-CORE\]](#) [service](#) entry that specifies where a [verifiable presentation](#) about the DID subject can be found. The [Decentralized Identity Foundation](#) hosts the [Linked VP Specification](#).

multibase

A specification for encoding binary data as a string using a prefix that indicates the encoding.

multikey

A verification method that encodes key types into a single binary stream that is then encoded as a [multibase](#) value.

multihash

Per the [\[MULTIFORMATS\]](#), [multihash](#) is a specification for differentiating instances of hashes. Software creating a hash prefixes (according to the specification) data to the hash indicating the algorithm used and the length of the hash, so that software receiving the hash knows how to verify it. Although [multihash](#) supports many hash algorithms, for interoperability, [DID Controllers](#) **MUST** only use the hash algorithms defined in this specification as permitted.

parameters

[did:webvh](#) parameters are a defined set of configurations that control how the issuer has generated the DID, and how the resolver must process the DID [Log entries](#). The use of parameters allows for the controlled evolution of [did:webvh](#) log handling, such as evolving the set of permitted hash algorithms or cryptosuites. This enables support for very long lasting identifiers – decades.

self-certifying identifier

An object identifier derived from initial data such that an attacker could not create a new object with the same identifier. The input for a [did:webvh](#) SCID is the initial [DIDDoc](#) with the placeholder [{SCID}](#) wherever the SCID is to be placed.

Verifiable Credential

A verifiable credential can represent all of the same information that a physical credential represents, adding technologies such as digital signatures, to make the credentials more tamper-evident and so more trustworthy than their physical counterparts. The [Verifiable Credential Data Model](#) is a W3C Standard.

Verifiable Presentation

A [verifiable presentation](#) data model is part W3C's [Verifiable Credential Data Model](#) that contains a set of [verifiable credentials](#) about a `credentialSubject`, and a signature across the verifiable credentials generated by that subject. In this specification, the use case of primary interest is where the DID is the `credentialSubject` and the DID signs the [verifiable presentation](#).

watcher

Watchers are entities within a decentralized trust ecosystem that monitor Decentralized Identifiers (DIDs) for changes or updates on behalf of their clients. Watchers maintain a historical cache of DID document versions and verify that the [DID Controller](#) is consistently following the prescribed evolution process. By ensuring integrity and traceability, watchers help foster trust among clients who rely on up-to-date and authentic DID information.

`did:webvh` watchers provide endpoints for retrieving DID information and to receive [webhooks](#) notifying the watcher about updates to the DID and deletion requests.

webhook

A webhook is a mechanism that enables real-time communication between systems by sending HTTP callbacks (typically POST requests) to a specified URL when an event occurs. Webhooks are commonly used for event-driven integrations and automation. Although webhooks are an implementation pattern rather than a formal standard, best practices are documented in [\[RFC8030\]](#).

witness

Witnesses are participants in the process of creating and verifying a version of a `did:webvh` [DIDDoc](#). Notably, a witness receives from the [DID Controller](#) a [DID](#) entry ready for publication, verifies it according to this specification, and approves it according to its ecosystem governance (whatever that might be). If the verification and approval process results are positive, witnesses returns to the DID Controller a [Data Integrity](#) proof attesting to that positive result.

threshold

An algorithm that defines when a sufficient number of [witnesses](#) have submitted valid [Data Integrity](#) proofs for a [DID Log entry](#) such that it is approved and can be published. The algorithm details are in the [Witness Threshold Algorithm](#) section of this specification.

W3C VCDM

A Verifiable Credential that uses the Data Model defined by the W3C [\[\[spec: W3C-VC\]\]](#) specification.

§ 7. References

DI-EDDSA-V1.0

[Data Integrity EdDSA Cryptosuites v1.0](#). Dave Longley; Manu Sporny; 2024-12-08. *Status: W3C Technical Recommendation*.

DID-CORE

[Decentralized Identifiers \(DIDs\) v1.0](#). Manu Sporny; Amy Guy; Markus Sabadello; Drummond Reed; 2022-07-19. *Status: REC.*

JSON-SCHEMA-CORE

[JSON Schema: A Media Type for Describing JSON Documents](#). A. Wright; H. Andrews; B. Hutton; G. Dennis; 2022-06-16. *Status: Internet Draft.*

MULTIFORMATS

[Multiformats](#). Juan Benet; Manu Sporny; 2024-02-21. *Status: Internet Draft.*

RFC1035

[Domain names - implementation and specification](#). P. Mockapetris; 1987-11. *Status: Internet Standard.*

RFC1123

[Requirements for Internet Hosts - Application and Support](#). R. Braden, Ed.; 1989-10. *Status: Internet Standard.*

RFC2181

[Clarifications to the DNS Specification](#). R. Elz; R. Bush; 1997-07. *Status: Proposed Standard.*

RFC2234

[Augmented BNF for Syntax Specifications: ABNF](#). D. Crocker, Ed.; P. Overell; 1997-11. *Status: Proposed Standard.*

RFC3491

[Nameprep: A Stringprep Profile for Internationalized Domain Names \(IDN\)](#). P. Hoffman; M. Blanchet; 2003-03. *Status: Proposed Standard.*

RFC3552

[Guidelines for Writing RFC Text on Security Considerations](#). E. Rescorla; B. Korver; 2003-07. *Status: Best Current Practice.*

RFC3912

[WHOIS Protocol Specification](#). L. Daigle; 2004-09. *Status: Draft Standard.*

RFC3986

[Uniform Resource Identifier \(URI\): Generic Syntax](#). T. Berners-Lee; R. Fielding; L. Masinter; 2005-01. *Status: Internet Standard.*

RFC4033

[DNS Security Introduction and Requirements](#). R. Arends; R. Austein; M. Larson; D. Massey; S. Rose; 2005-03. *Status: Proposed Standard.*

RFC4034

[Resource Records for the DNS Security Extensions](#). R. Arends; R. Austein; M. Larson; D. Massey; S. Rose; 2005-03. *Status: Proposed Standard*.

RFC4035

[Protocol Modifications for the DNS Security Extensions](#). R. Arends; R. Austein; M. Larson; D. Massey; S. Rose; 2005-03. *Status: Proposed Standard*.

RFC6125

[Representation and Verification of Domain-Based Application Service Identity within Internet Public Key Infrastructure Using X.509 \(PKIX\) Certificates in the Context of Transport Layer Security \(TLS\)](#). P. Saint-Andre; J. Hodges; 2011-03. *Status: Proposed Standard*.

RFC6234

[US Secure Hash Algorithms \(SHA and SHA-based HMAC and HKDF\)](#). D. Eastlake 3rd; T. Hansen; 2011-05. *Status: Informational*.

RFC6973

[Privacy Considerations for Internet Protocols](#). A. Cooper; H. Tschofenig; B. Aboba; J. Peterson; J. Morris; M. Hansen; R. Smith; 2013-07. *Status: Informational*.

RFC8030

[Generic Event Delivery Using HTTP Push](#). M. Thomson; E. Damaggio; B. Raymor, Ed.; 2016-12. *Status: Proposed Standard*.

RFC8484

[DNS Queries over HTTPS \(DoH\)](#). P. Hoffman; P. McManus; 2018-10. *Status: Proposed Standard*.

RFC8785

[JSON Canonicalization Scheme \(JCS\)](#). A. Rundgren; B. Jordan; S. Erdtman; 2020-06. *Status: Informational*.

RFC9110

[HTTP Semantics](#). R. Fielding, Ed.; M. Nottingham, Ed.; J. Reschke, Ed.; 2022-06. *Status: Internet Standard*.

RFC9233

[Internationalized Domain Names for Applications 2008 \(IDNA2008\) and Unicode 12.0.0](#). P. Fältström; 2022-03. *Status: Proposed Standard*.

RFC9457

[Problem Details for HTTP APIs](#). M. Nottingham; E. Wilde; S. Dalal; 2023-07. *Status: Proposed Standard*.

RFC9525

[Service Identity in TLS](#). P. Saint-Andre; R. Salz; 2023-11. *Status: Proposed Standard*.

§ 8. **did:webvh** Version Changelog

The following lists the substantive changes in each version of the specification.

- Version 1.0
 - Adds clarifications about the handling of default values for unspecified parameters when introduced, and defines that **null** should **NOT** be used for parameters, to ensure that the parameter data types are always retained.
 - Removes the **weight** value and clarifies permitted values of **threshold**.
 - Adds the concept of a **watcher** to the specification, including the technical details of deploying and using a **watcher**.
 - Clarifies the [DID-to-HTTPS Transformation](#) to include support for domain names using Unicode international languages.
- Version 0.5
 - Remove the **prerotation** parameter. The feature is automatically enforced when **nextKeyHashes** is present.
 - Clarify the way the [Pre-Rotation](#) feature works, once a **nextKeyHashes** is committed, the next [DID log entries](#) has to be signed by one of the committed keys.
 - Clarify how to stop using [pre-rotation](#), including when deactivating the DID.
 - Change the [witness](#) handling by removing the witness [Data Integrity](#) proofs from the [DID Log](#) file and puts them into a separate file **did-witness.json**. Adjustments to the witness threshold algorithm were also made, such as removing the [DID Controllers'](#) **selfweight** attribute, and defining that all witness DIDs must be **did:key** DIDs.
 - Clarify how to stop using witnesses.
 - Clarify the initialization values of [parameters](#) and that array [parameters](#) must use **null** and **not** use empty lists (**[]**) when not active.
 - Rename the DID Method to **did:webvh** (**did:web** + Verifiable History)
 - Move the DID Method information site to <https://didwebvh.info>.
- Version 0.4
 - Removes large non-normative sections, such as the implementer's guide, as they are now published on the <https://didtdw.org/> information site.
 - Removes the use of JSON Patch from the specification. The full DIDDoc is included in each [DID log entry](#).

- Changes the data format of the [DID log entries](#) from an array to an object. The [DID Log](#) remains in the [JSON Lines](#) format.
- Changes the [DID log entry](#) array to be named JSON objects or properties.
- Makes each DID version's [Data Integrity](#) proof apply across the JSON [DID log entry](#) object, as is typical with [Data Integrity](#) proofs. Previously, the [Data Integrity](#) proof was generated across the current DIDDoc version, with the `versionId` as the challenge.
- Specified that the `versionTime` must be recorded as a UTC time zone timestamp.
- Version 0.3
 - Removes the `cryptosuite parameter`, moving it to implied based on the `method parameter`.
 - Change base32 encoding with [base58btc](#), as it offers a better expansion rate.
 - Remove the step to extract part of the [base58btc](#) result during the generation of the [SCID](#).
 - Use [multihash](#) in the [SCID](#) to differentiate the different hash function outputs.
- Version 0.2
 - Changes the location of the [SCID](#) in the DID to always be the first component after the DID Method prefix – `did:tdw:<scid>:...`
 - Adds the `parameter portable` to enable the capability to move a `did:tdw` during the creation of the DID.
 - Removes the first two [Log Entry](#) items `entryHash` and `versionId` and replacing them with the new `versionId` as the first item in each [log entry](#). The new `versionId` takes the form `<version number>-<entryHash>`, where `<version number>` is the incrementing integer of version of the entry: 1, 2, 3, etc.
 - The `<did>/whois` media type is changed to `application/vp` and the file is changed to `whois.vp` to match the IANA registration of a [Verifiable Presentation](#).